

SC1006: Computer Organisation and Architecture

Weeks 8–9 — Signal Chain, Data Transfer & Memory Sub-Systems

Contents

9	Signal Chain & Data Transfer Sub-Systems	2
9.1	Signal Chain Sub-System	2
9.1.1	Overview of Computer Sub-Systems	2
9.1.2	Signals: Analog and Digital	2
9.1.3	Analog-to-Digital and Digital-to-Analog Conversion	2
9.1.4	Electrical Signal Interfaces	3
9.2	Parallel Data Transfer	4
9.2.1	Concept and Operation	4
9.2.2	Signal Skew	4
9.2.3	Crosstalk	5
9.2.4	Advantages and Disadvantages of Parallel Transfer	5
9.3	Serial Data Transfer	5
9.3.1	Concept and Motivation	5
9.3.2	Transfer Direction Modes	6
9.3.3	Synchronous vs. Asynchronous Serial Transfer	6
9.3.4	Serial Peripheral Interface (SPI) Bus	7
9.3.5	Universal Asynchronous Receiver Transmitter (UART)	7
9.4	Interrupts and Polling	8
9.4.1	Data Transfer Mechanism Overview	8
9.4.2	Polling	9
9.4.3	Interrupts	9
9.5	Direct Memory Access (DMA)	10
9.5.1	Motivation	10
9.5.2	Basic DMA Process	11
9.5.3	DMA Transfer Modes	11
9.5.4	Comparison of DMA Transfer Modes	12
10	Memory Sub-System	13
10.1	Memory Sub-System: Overview	13
10.1.1	Role of Memory in a Computer System	13
10.1.2	Functional Classification: System Memory vs. Storage Memory	13
10.1.3	The Memory Hierarchy	14
10.2	Volatile Memory: SRAM and DRAM	14
10.2.1	Static RAM (SRAM)	14
10.2.2	Dynamic RAM (DRAM)	15
10.3	Non-Volatile Memory	16
10.3.1	EPROM and EEPROM	16
10.3.2	The Floating Gate Transistor (FGT)	16

10.3.3	Flash Memory	17
10.4	Storage Devices: HDD and SSD	18
10.4.1	Magnetic Hard Disk Drive (HDD)	18
10.4.2	Solid State Drive (SSD)	20
10.5	Data Centre Storage	21
10.5.1	Storage Selection Criteria for Data Centres	21
10.5.2	Why HDDs Still Dominate Data Centre Storage	22
10.5.3	Heat Assisted Magnetic Recording (HAMR)	22
11	Cache Memory	24
11.1	Motivation and Overview	24
11.1.1	CPU Memory Access Sequence	24
11.2	Principle of Locality	25
11.3	Cache Mapping Scheme	25
11.3.1	Key Terminology	25
11.3.2	Direct Mapped Cache	26
11.3.3	Cache Hit and Miss Detection	27
11.4	Cache Replacement Policy	27
11.5	Cache Write Policy	27
11.5.1	Write-Hit Policies	27
11.5.2	Write-Miss Policies	28
11.6	Effective Access Time (EAT)	28
12	Virtual Memory Management	30
12.1	Memory Address Space	30
12.1.1	Virtual vs. Physical Address	30
12.2	Virtual Memory Management	30
12.2.1	Advantages of Virtual Memory	31
12.3	Paging: Address Mapping Scheme	31
12.3.1	The Page Table	31
12.4	Virtual-to-Physical Address Translation	32
12.4.1	Address Field Partitioning	32
12.4.2	Worked Example: Paging System	32
12.5	Integration of Cache and Virtual Memory	33
12.6	Translation Lookaside Buffer (TLB)	34
12.6.1	TLB Lookup: Three Scenarios	34
13	Communication and User Interface	36
13.1	Wired Communication Interfaces	36
13.1.1	Universal Serial Bus (USB)	36
13.1.2	High-Definition Multimedia Interface (HDMI)	37
13.2	Wireless Communication	37
13.2.1	ISM Radio-Frequency Bands	37
13.2.2	Wi-Fi (IEEE 802.11)	38
13.2.3	Bluetooth	38
13.2.4	Factors Affecting Wireless Transmission Range	38
13.2.5	Mitigating Interference	38
13.3	User Interface Devices — Input	39
13.3.1	Keyboard	39
13.3.2	Mouse	39
13.3.3	Capacitive Touch Interface	39
13.4	User Interface Devices — Sensors	40

13.4.1	Camera	40
13.4.2	Microphones	40
13.5	User Interface Devices — Output	41
13.5.1	Liquid Crystal Display (LCD)	41
13.5.2	Audio Speakers	41
14	Computer Arithmetic and Number Representation	43
14.1	Signed Integer Representation	43
14.1.1	Positive and Negative Numbers in Binary	43
14.1.2	Two's Complement Representation	43
14.1.3	Carry vs. Overflow	44
14.1.4	Sign Extension	44
14.1.5	Multi-Precision Arithmetic	45
14.2	Fixed-Point and Floating-Point Number Systems	45
14.2.1	Range and Precision	45
14.2.2	Fixed-Point Representation	45
14.2.3	Floating-Point Representation	46
14.3	IEEE 754 Floating-Point Standard	46
14.3.1	Normalised Numbers	47
14.3.2	Representable Range (Single Precision)	47
14.3.3	IEEE 754 Encoding Summary	47
14.4	Fixed-Point vs. Floating-Point Comparison	48
14.5	Computer Arithmetic Considerations in Coding	48
14.5.1	Effects of the Four Arithmetic Operators	48
14.5.2	Effects of Rounding	49
14.5.3	Handling Numbers with Different Magnitudes	49
14.5.4	Rules of Thumb for Maximising Accuracy	50
15	Processor Pipeline	51
15.1	Review: Sequential Instruction Execution	51
15.2	Pipeline Architecture	51
15.2.1	Basic Concept	51
15.2.2	Pipeline Efficiency	52
15.3	Pipeline Conflicts	52
15.3.1	Resource Conflict (Structural Hazard)	52
15.3.2	Data Dependency Conflict (Data Hazard)	53
15.4	Branch Conflicts (Control Hazards)	54
15.4.1	Branch Instruction Behaviour	54
15.4.2	Reducing Branch Delay	54
15.4.3	Delayed Branching	54
15.4.4	Dynamic Branch Prediction	55
15.5	Note on ARM Instructions in Pipeline Examples	56
15.6	Real-World Connection: ARM Cortex-M3/M4	56

Chapter 9

Signal Chain & Data Transfer Sub-Systems

9.1 Signal Chain Sub-System

9.1.1 Overview of Computer Sub-Systems

A modern computer is most naturally understood by decomposing it into a set of cooperating sub-systems. The three principal groups covered in the second half of SC1006 are:

- **Signal Chain Sub-System** — ADC/DAC conversion, parallel and serial digital interfaces, the Direct Memory Access (DMA) controller, and the Interrupt Controller.
- **Memory Sub-System** — SRAM, DRAM, Flash, SSDs, HDDs, Cache, and Virtual Memory.
- **Communication and User Interface Sub-System** — Wired (USB, SATA, HDMI), Wireless (Wi-Fi, Bluetooth), input and output peripherals.

This chapter focuses on the signal chain, which bridges the real world (inherently analog) and the digital processor.

9.1.2 Signals: Analog and Digital

Definition 9.1.1 (Signal). A *signal* is any real-valued function that varies in time and/or space. Examples include audio waveforms, electrocardiograms (ECG), and pulse trains.

All real-world physical quantities—sound, light, heat, pressure—are *analog*: their magnitude varies continuously both in value and in time. Historically, processing was carried out entirely in the analog domain (e.g., analog equalizers, TV brightness control). Over the past 40 years, digital processors have become sufficiently economical and flexible to displace analog circuitry as the dominant processing platform, primarily because digital representations are highly tolerant of noise and component aging.

9.1.3 Analog-to-Digital and Digital-to-Analog Conversion

Since a digital processor operates exclusively on discrete binary values, any analog real-world signal must be *digitized* before processing and *reconstructed* afterward.

9.1.3.1 Sampling and Quantization

The analog signal is sampled at discrete time intervals called the *sampling interval* T_s . At each sampling point the instantaneous signal level is mapped to the nearest discrete voltage level permitted by the digitization process. For an n -bit ADC there are 2^n discrete output levels (e.g., a 3-bit ADC has levels 000₂ through 111₂, i.e. 8 levels).

Key Result: Nyquist Sampling Theorem

The sampling frequency f_s must satisfy

$$f_s \geq 2 f_{\max},$$

where $f_{\max}$ is the highest frequency component present in the analog signal. Sampling below this rate causes *aliasing* — the reconstructed signal will be incorrect.

9.1.3.2 Digital-to-Analog Reconstruction

After digital processing, the DAC converts the discrete output sequence back to an analog signal. A *smoothing (low-pass) filter* is applied post-DAC to interpolate between the discrete output steps and recover a continuous waveform.

9.1.4 Electrical Signal Interfaces

Definition 9.1.2 (Interface). An *interface* is the boundary at which two or more devices meet to exchange information. Data may be transferred serially (one bit at a time on a single line) or in parallel (multiple bits simultaneously on a bus), and each mode may be unidirectional or bidirectional.

For two devices to communicate correctly, two conditions must be satisfied simultaneously:

1. **Electrical signal level compatibility** — the voltage levels used by each device must be mutually recognizable.
2. **Communication protocol compatibility** — both devices must speak the same handshaking and data protocol.

9.1.4.1 Logic Level Parameters

Digital data is encoded as logic 0 or 1 via voltage levels. Four device parameters define whether two devices can communicate:

Parameter	Definition
VOH	Minimum output voltage guaranteed for a transmitted logic 1
VOL	Maximum output voltage guaranteed for a transmitted logic 0
VIH	Minimum input voltage that the receiver recognizes as logic 1
VIL	Maximum input voltage that the receiver recognizes as logic 0

Key Result: Logic Level Compatibility Conditions

For device A (transmitter) and device B (receiver) to communicate without error:

$$VOH_A \geq VIH_B \quad \text{and} \quad VOL_A \leq VIL_B.$$

The margins $(VOH_A - VIH_B)$ and $(VIL_B - VOL_A)$ represent the *noise margin* of the link.

Warning: Voltage Mismatch Can Destroy Hardware

Connecting a 5 V output device to a 3 V input device without level-shifting circuitry will exceed the maximum rated input voltage of the receiving device, causing permanent damage (“fried” IC) or degraded reliability. Always verify voltage ratings from device datasheets before connecting two devices.

9.1.4.2 Differential Signaling

In single-ended signaling the data voltage is measured relative to a common ground. In *differential signaling*, the signal is carried on a wire pair (V^+ , V^-) where V^- is the logical inversion of V^+ :

$$\text{Logic 1 : } V^+ > V^- \quad \text{Logic 0 : } V^- > V^+$$

Differential signaling offers two key advantages over single-ended signaling:

- **Higher noise margin** — the effective swing between 0 and 1 is doubled ($2\Delta V$ rather than ΔV).
- **Common-mode noise rejection** — any external interference couples equally onto V^+ and V^- , leaving their *difference* unchanged and therefore canceling noise.

These properties allow differential buses (e.g., USB, SATA, HDMI) to operate at much higher clock frequencies and over longer cable runs than single-ended buses.

9.2 Parallel Data Transfer

9.2.1 Concept and Operation

In a *parallel interface*, N data bits are driven simultaneously on N separate signal lines (a *data bus*), plus at least one *strobe* (clock) line. The receiver latches all N bits together on a defined edge of the strobe signal (e.g., the rising edge).

Definition 9.2.1 (Parallel Data Transfer). A data transfer scheme in which multiple bits are transmitted simultaneously over dedicated signal lines, synchronized by a common strobe signal. Examples include SRAM, DRAM, and ROM buses.

Given the same clock frequency f , a parallel bus with width N achieves a raw data rate of $N \times f$ bits per second — substantially higher than a single-bit serial link.

9.2.2 Signal Skew

Definition 9.2.2 (Signal Skew). *Signal skew* is the difference in arrival times of two signals that are intended to be simultaneous. In a parallel bus, skew arises when individual data lines have different propagation delays.

9.2.2.1 Propagation Delay and the RC Model

The propagation delay of a signal line is governed primarily by its resistance R and capacitance C . The time constant

$$\tau = RC$$

determines how quickly a voltage transition propagates to its final level. Differences in R or C between lines (caused by variations in PCB trace length, trace width, or the loading of active components) produce *skew*.

Example 9.2.3. Suppose a 2-bit bus transmits $D_1D_0 = 01$ (i.e., $D_1 = 0$, $D_0 = 1$). If the D_0 line has higher capacitance and its signal is delayed past the strobe edge, the receiver will sample $D_0 = 0$ (the old value) rather than 1, incorrectly latching 00 instead of 01.

Exam Tip: Skew is the Key Limiter of Parallel Bus Speed

In parallel bus design, all traces of the same bus must be *length-matched* on the PCB so that their RC delays are equal. Failure to do so limits the maximum safe clock frequency and transfer distance.

9.2.3 Crosstalk

Definition 9.2.4 (Crosstalk). *Crosstalk* is the undesired coupling of a signal from one conductor onto an adjacent conductor in the same bus or cable.

Crosstalk arises through two mechanisms:

- **Capacitive (electric field) coupling** — a rapidly changing voltage on one trace induces a charge on neighboring traces via inter-trace capacitance.
- **Inductive (magnetic field) coupling / electromagnetic radiation** — current-carrying traces act as antennas and radiate signals that are picked up by neighboring traces.

The tight packing of many signal lines in a parallel cable or PCB increases the probability of both mechanisms.

9.2.4 Advantages and Disadvantages of Parallel Transfer

Advantages	Disadvantages
High raw data rate: N bits per clock cycle.	Signal skew limits maximum clock frequency and cable length.
	Crosstalk increases with bus width and clock frequency.
	More wires $\Rightarrow$ bulkier cables and larger PCB routing area.
	Higher hardware cost per interface.

9.3 Serial Data Transfer

9.3.1 Concept and Motivation

In a *serial interface* data bits are transmitted one at a time over a single data line per direction. With only one (or two) data wires, serial buses are far less susceptible to signal skew and crosstalk than parallel buses. This makes it possible — with careful design — to operate a serial bus at a *higher clock frequency* than a comparable parallel bus, partially compensating for the lower per-clock data rate.

Definition 9.3.1 (Serial Data Transfer). A data transfer scheme in which bits are sent sequentially over a single data line, one bit per clock cycle. The receiver must perform serial-to-parallel conversion to reconstruct the original word before handing it to the processor.

9.3.1.1 Pros and Cons

Advantages	Disadvantages
Less affected by skew and crosstalk (fewer wires).	Lower raw data rate than parallel at the same clock frequency.
Supports longer cable runs.	Hardware must perform serialization and deserialization.
Lower connector and cabling cost.	

9.3.2 Transfer Direction Modes

Definition 9.3.2 (Simplex). Data flows in one direction only. One device is always the transmitter; the other is always the receiver.

Definition 9.3.3 (Half-Duplex). Both devices can transmit and receive, but not simultaneously. A single shared data line is used; at any moment, only one device transmits.

Definition 9.3.4 (Full-Duplex). Both devices transmit and receive simultaneously over two separate data lines (one per direction).

9.3.3 Synchronous vs. Asynchronous Serial Transfer

9.3.3.1 Synchronous Transfer

In synchronous transmission a *common clock signal* is shared between transmitter and receiver. This clock — provided by the *master* in a master-slave configuration — tells the receiver exactly when to sample each incoming data bit. No bandwidth is consumed for synchronization overhead, enabling potentially higher throughput.

Key Result: Synchronous Serial — Master Provides the Clock

In a synchronous serial link the master device drives the clock line. Data is latched by the slave on a configurable edge (rising or falling). Because timing is explicit, synchronous links require no embedded synchronization overhead.

9.3.3.2 Asynchronous Transfer

In asynchronous transmission there is no shared clock line. Synchronization is instead achieved by:

1. Both devices agree in advance on a common *baud rate* (bits per second) and data frame format.
2. The transmitter embeds *START* and *STOP* synchronization words/bits in the data stream.
3. Upon detecting the *START* condition, the receiver activates its internal clock (running at the agreed baud rate) to track subsequent bits.

A fundamental challenge is *clock drift*: since the two devices use independent local oscillators, their clocks will gradually diverge. To mitigate drift, the transmitter may send periodic SYNC words so the receiver can realign its clock.

Warning: Baud Rate Mismatch Corrupts Data

If the receiver baud rate differs from the transmitter baud rate, the receiver will sample at the wrong time instants, producing incorrect data. In a UART link operating at 100 000 bps from the transmitter, a receiver clocked at 200 000 bps will take double the number of samples and decode entirely wrong bits.

9.3.4 Serial Peripheral Interface (SPI) Bus

SPI is a widely used *synchronous* serial bus standard. It uses four signals:

Signal	Function
SCLK	Serial clock (driven by master)
MOSI	Master Out, Slave In — master → slave data
MISO	Master In, Slave Out — slave → master data
SS	Slave Select — pulled low by master to select the target slave

Multiple slave devices can be connected to a single master by providing one dedicated **SS** line per slave. A transfer begins when the master asserts **SS** low; data on **MOSI** and **MISO** is latched on each clock edge (rising or falling, configurable). Because SPI is full-duplex, master and slave exchange data simultaneously.

Exam Tip: SPI Signal Names

Remember: **MOSI** is from the *master's* perspective (master outputs, slave reads). **MISO** is the slave's output received by the master. The clock is *always* driven by the master regardless of data direction.

9.3.5 Universal Asynchronous Receiver Transmitter (UART)

UART is one of the most widely used *asynchronous* serial protocols and forms the basis of the RS-232 COM port standard. Modern development boards (Arduino, TIVA-C, Raspberry Pi) commonly expose a UART interface, often enumerated as a Virtual COM Port over USB.

9.3.5.1 UART Frame Format

A single UART data frame consists of the following fields transmitted in order:

1. **IDLE state** — the data line is held at logic 1.
2. **START bit** — logic 0; the falling edge alerts the receiver.
3. **Data bits** — 5–9 bits, transmitted *LSB first*.
4. **Parity bit** (optional) — used for basic error detection.
5. **STOP bit(s)** — logic 1; restores the line to IDLE.

Key Result: UART Fixed Logic Levels

- IDLE = 1
- START bit = 0
- STOP bit = 1

These are fixed and not configurable.

The UART configuration is described by a shorthand such as 8N1 (8 data bits, No parity, 1 STOP bit) or 7O1 (7 data bits, Odd parity, 1 STOP bit).

9.3.5.2 Parity

The optional parity bit provides single-bit error *detection* (not correction):

- **Even parity** — the transmitter sets the parity bit such that the total number of 1s in the data field plus parity field is even.
- **Odd parity** — similarly, the total number of 1s is odd.
- **No parity** — the parity bit is omitted entirely.

If the receiver counts a number of 1s inconsistent with the agreed parity scheme, it flags a *Parity Error*. If the STOP bit is detected as 0, a *Framing Error* is flagged.

Example 9.3.5 (UART 7O1 Transmission of ASCII ‘J’). ASCII ‘J’ = $0x4A = 1001010_2$. Transmitted LSB-first: 0101001. With odd parity, there are 3 ones in the data field; the parity bit is set to 0 so that the total count of ones (3) remains odd.

$\underbrace{0}_{\text{START}} \quad \underbrace{0101001}_{\text{Data (LSB first)}} \quad \underbrace{0}_{\text{Parity}} \quad \underbrace{1}_{\text{STOP}}$

9.3.5.3 UART Receiver Operation

Because there is no explicit clock, the UART receiver runs its internal clock at a multiple of the baud rate (typically $16\times$). Upon detecting the START bit’s falling edge, the receiver uses this faster clock to estimate the *mid-point* of each data bit, where the signal is most stable, and samples there. This technique is critical for reliable reception.

Exam Tip: UART Receiver Internal Clock

The UART receiver internal clock runs at $16\times$ baud rate (in real-world designs) to locate the mid-point of each bit for sampling. In exam questions a simplified $4\times$ model may be used for timing diagram analysis.

9.4 Interrupts and Polling

9.4.1 Data Transfer Mechanism Overview

The CPU, memory, and I/O peripherals are interconnected via a *system bus*. Three mechanisms govern how data is transferred across this bus:

1. **Polling (Programmed I/O)** — CPU actively and continuously checks the I/O device status.
2. **Interrupt-Triggered I/O** — the I/O device signals the CPU only when it needs attention.

3. **Direct Memory Access (DMA)** — a dedicated controller moves data independently of the CPU (covered in Chapter 9.5).

9.4.2 Polling

Definition 9.4.1 (Polling / Programmed I/O). A data transfer mechanism in which the CPU repeatedly queries the status register of an I/O device in a software loop, waiting until the device is ready before initiating the transfer.

The control flow is:

1. Initialise control registers for the transfer.
2. Read the device status register.
3. If device is not ready, return to step 2 (*busy-wait loop*).
4. If device is ready, perform the data transfer and exit the loop.

Advantages	Disadvantages
Programmer retains full, explicit control.	CPU is 100% occupied and cannot execute any other code during the wait.
Simplest to implement and debug.	Highly inefficient when I/O device response is slow or unpredictable.

9.4.3 Interrupts

Definition 9.4.2 (Interrupt). An *interrupt* is a signal — either an electrical pulse on a dedicated pin or a change in a special-purpose register — that alerts the CPU that an event requires its attention. The CPU may choose to service or ignore the request.

Analogy: Polling is like watching a pot of water continuously to see when it boils. Interrupts are like a kettle with a whistle — you are free to do other work until the device signals that it is ready.

9.4.3.1 Interrupt Control Flow

When an interrupt request is received and the CPU decides to service it:

1. CPU suspends the current program at the end of the current instruction.
2. CPU looks up the *Interrupt Vector Table* (IVT) to find the starting address of the *Interrupt Service Routine* (ISR) for this interrupt source.
3. CPU saves the *processor context* (key register values, program counter) onto the stack, so the interrupted program can resume correctly.
4. CPU branches to and executes the ISR.
5. On ISR completion, CPU restores the saved context and resumes the interrupted program from the point it was suspended.

Definition 9.4.3 (Interrupt Latency). The time delay between the CPU *receiving* an interrupt request and the CPU *beginning to execute* the first instruction of the corresponding ISR.

Key Result: Interrupt Control Flow Summary

Interrupt → Suspend main program → Save context → Look up IVT → Execute ISR → Restore context → Resume main program.

9.4.3.2 Interrupt Vector Table

The *Interrupt Vector Table* (IVT) is a data structure stored in memory that maps each interrupt source to the starting address of its ISR. Each interrupt is assigned a unique index into this table; the CPU uses this index to locate the correct ISR address without needing to search.

9.4.3.3 Interrupt Service Routine Design

Because the main program is suspended while the ISR executes, ISRs should be kept *short* to minimise the disruption to the interrupted task. When multiple interrupts arrive simultaneously, the system uses a *priority* or *first-come-first-served* arbitration scheme to determine which ISR runs first.

9.4.3.4 Pros and Cons of Interrupt-Driven I/O

Advantages	Disadvantages
CPU is free to execute other tasks between interrupts.	Additional hardware circuitry required for the interrupt controller.
Efficient CPU utilization.	Harder to debug than polling due to asynchronous, non-deterministic firing.
Supports task priority and pre-emption (essential for OS design).	

Exam Tip: Polling vs. Interrupt Trade-offs

For exam questions comparing the two mechanisms: Polling gives *deterministic timing* and simplicity at the cost of CPU efficiency. Interrupts give *CPU efficiency and responsiveness* at the cost of implementation complexity and harder debugging. DMA (Chapter 9.5) goes further — it removes the CPU from the data movement entirely.

9.5 Direct Memory Access (DMA)

9.5.1 Motivation

Both polling and interrupt-driven I/O require the CPU to execute the actual data transfer instructions. As the volume of data increases (e.g., HDD transfers, audio streaming), the CPU spends an increasing fraction of its time simply moving data from one address to another — time that could otherwise be devoted to algorithm processing.

Definition 9.5.1 (DMA Controller (DMAC)). A *Direct Memory Access Controller* is a hardware module capable of controlling the system bus and performing data transfers between memory and memory, memory and I/O peripherals, or between two I/O peripherals, *without CPU involvement* once configured.

A DMAC includes dedicated address-generation hardware that can efficiently handle complex access patterns (e.g., de-interleaving left and right channel audio data) more efficiently than a general-purpose CPU.

9.5.2 Basic DMA Process

The DMA transfer involves the following steps:

1. **CPU configures DMAC** with:
 - Source address
 - Destination address
 - Number of data words (or bytes) to transfer
 - Trigger signal (DMAREQ, software command, etc.)
2. **CPU resumes normal execution.** DMAC enters IDLE state waiting for the trigger.
3. **Trigger fires.** DMAC requests ownership of the system bus.
4. **DMAC transfers data** according to its configuration.
5. **Transfer completes.** DMAC asserts a *DMA interrupt* to the CPU, releases the bus, and the CPU resumes processing on the newly transferred data.

Key Result: DMA Bus Arbitration

The CPU and DMAC share the system bus. When both need the bus simultaneously, the higher-priority unit is granted access. Priority assignment (fixed or configurable) is processor-specific. If DMAC has lower priority, it waits; if DMAC has higher priority, the CPU is stalled.

9.5.3 DMA Transfer Modes

9.5.3.1 Burst Mode

Definition 9.5.2 (Burst Mode). The DMAC seizes the system bus and transfers *multiple data units* (possibly the entire block) before releasing control.

- **Timing:** DMAC performs a Setup phase (bus request), transfers all data (source → DMAC buffer → destination), then releases the bus.
- **CPU impact:** CPU may be stalled for the full duration of the burst if it needs the bus.
- **Use case:** Applications with bursty large transfers (e.g., HDD file transfer) where CPU responsiveness is less critical.

9.5.3.2 Cycle Stealing

Definition 9.5.3 (Cycle Stealing). The DMAC seizes the bus, transfers *one unit of data*, then immediately releases the bus back to the CPU. It waits for a minimum hold-off period before requesting the bus again.

- **Timing:** Repeated short sequences of: Setup → transfer one unit → Release → wait → repeat.
- **CPU impact:** CPU is only stalled for the duration of one transfer unit at a time, keeping it responsive.
- **Use case:** Real-time applications that require the CPU to remain responsive (e.g., real-time security monitoring, audio processing).

The DMAC typically schedules cycle-stealing transfers *between CPU instructions* or *between pipeline stages* to minimize pipeline disruption.

9.5.3.3 Transparent Mode

Definition 9.5.4 (Transparent Mode). The DMAC transfers data *only when the CPU is not using the system bus*. The CPU experiences zero performance degradation from bus conflicts.

- **CPU impact:** Effectively zero (the DMAC never competes with the CPU for the bus).
- **Data rate:** Potentially the slowest of the three modes, and highly variable, since it is limited by the fraction of time the bus is idle.
- **Hardware complexity:** Requires dedicated logic to continuously monitor bus activity.

9.5.4 Comparison of DMA Transfer Modes

	Burst	Cycle Stealing	Transparent
Data transfer rate	Highest	Medium	Lowest (variable)
CPU stall duration	Longest	Short	None
CPU responsiveness	Lowest	Medium	Highest
Hardware complexity	Moderate	Moderate	Highest
Typical use case	HDD bulk transfers	Real-time systems	Background transfers

Key Result: DMA Mode Selection

- Use **Burst** when you need maximum throughput and CPU responsiveness is not a concern.
- Use **Cycle Stealing** when the CPU must remain responsive while data transfer is ongoing.
- Use **Transparent** when any CPU performance impact is unacceptable and transfer rate can be sacrificed.

Exam Tip: Timing Diagram Calculations for DMA

For timing questions involving Burst or Cycle Stealing mode, you need to account for:

- **Setup time:** time for DMAC to request and obtain the bus.
- **Transfer time:** time to move one data unit from source → DMAC buffer → destination (two sub-phases in Fetch-and-Deposit architecture).
- **Release time:** time for DMAC to return bus control to the CPU.
- **Hold-off / wait time** (Cycle Stealing): minimum time DMAC must wait between bus requests.

These timing parameters are given in the question; compute total transfer time and CPU stall time from the sum of the relevant phases.

Chapter 10

Memory Sub-System

10.1 Memory Sub-System: Overview

10.1.1 Role of Memory in a Computer System

Recall from Section 9.1.1 that a modern computer is decomposed into three cooperating sub-systems. Having covered the Signal Chain sub-system (Chapter 9.1) and the Data Transfer sub-system (Chapters 9.2–9.5), we now turn to the *Memory Sub-System*. Computer memory is a foundational component: it is the location where all program code and data are stored and from which the processor fetches instructions to execute.

There is a rich variety of memory technologies available — semiconductor-based devices (SRAM, DRAM, Flash, SSD) alongside magnetic storage (HDD). A key question when surveying these technologies is: *which are volatile and which are non-volatile?*

Definition 10.1.1 (Volatile Memory). A *volatile* memory loses its stored data when electrical power is removed. It is therefore used as temporary (runtime) storage. Examples: SRAM, DRAM.

Definition 10.1.2 (Non-Volatile Memory). A *non-volatile* memory retains its stored data even after power is removed. It is used as permanent storage. Examples: Flash, HDD, SSD.

10.1.2 Functional Classification: System Memory vs. Storage Memory

From a functional perspective, every memory element in a computer falls into one of two categories:

- **System Memory** — holds the runtime program code and data actively used by the processor. The processor executes instructions *directly* from system memory (a property called *Execute In Place*, XIP). For this course, the three memory types that support XIP are **SRAM**, **DRAM** (via a hardware DRAM controller that handles the multi-step access transparently), and **NOR Flash**.
- **Storage Memory** — holds the complete collection of all program code and data in the system. Code cannot be executed directly from storage memory; it must first be transferred to system memory. Any memory type may serve as storage memory; however, since storage memory must preserve data across power cycles, **non-volatile** memory is required in practice.

Key Result: System Memory Must Support XIP

Only memories that allow random read access using a simple address (SRAM, DRAM with controller, NOR Flash) can serve as system memory. NAND Flash and HDD do *not* support XIP and can only be used as storage memory.

10.1.3 The Memory Hierarchy

Different memory technologies trade off *speed*, *cost*, and *capacity* differently. Organising them in a pyramid places the fastest (and most expensive) memories at the apex and the slowest (but cheapest) at the base:

Level	Technology	Characteristics
Registers / Cache	SRAM	Fastest, most expensive, smallest
Main Memory	DRAM	Fast, moderate cost, GByte range
System Flash	NOR Flash	Slower, non-volatile, supports XIP
Mass Storage	NAND Flash / SSD	High density, limited endurance
Bulk Storage	HDD	Cheapest per bit, mechanical, slow

The access time figures vary significantly with advances in semiconductor technology; the hierarchy of relative speeds, however, is a persistent architectural principle.

10.2 Volatile Memory: SRAM and DRAM

10.2.1 Static RAM (SRAM)

Definition 10.2.1 (SRAM). *Static Random Access Memory* (SRAM) stores data as long as power is applied. Unlike DRAM, it requires no periodic refresh. It is implemented with 4–6 transistors per cell (typically 6), making it larger and more expensive per bit than DRAM, but also significantly faster.

10.2.1.1 SRAM Cell Structure

A standard SRAM bit cell consists of **six transistors** (M1–M6):

- **M1–M4** form two cross-coupled inverters. Their outputs are connected to each other's inputs, creating a stable bistable latch: the cell holds its logic state indefinitely as long as power is supplied.
- **M5 and M6** are pass (switch) transistors controlled by the *word line* (WL). When the WL is asserted (driven high by the address decoder), these transistors connect the cell to the *bit lines* (BL and $\overline{\text{BL}}$), enabling a read or write operation.

10.2.1.2 SRAM Read and Write Operations

To interface with an SRAM chip, the following signal lines are used:

Signal	Function
Addr	Address of the target memory location
CS	Chip Select — master enable for the chip
WE	Write Enable — signals a write operation
OE	Output Enable — allows the chip to drive the data bus
Data	Bidirectional data bus (typically 8, 16, or 32 bits)

Read sequence: The processor outputs the address; then asserts CS and OE. The SRAM decodes the address, enables the target word line, and drives the data onto the bus via the sense amplifier.

Write sequence: The processor outputs the address, then asserts CS and WE. Once the data is placed on the data bus, the SRAM latches it into the cell.

Exam Tip: SRAM Timing Signals

In timing diagrams, the address is always presented *before* CS/OE or CS/WE are asserted, because the chip needs a stable address to decode before it can activate the correct word line. For read, OE gates the output; for write, WE gates the input latch.

10.2.1.3 SRAM Internal Architecture

Internally, an SRAM chip contains:

- A **memory array** of SRAM cells organized in rows and columns (e.g., 2^{20} cells for a 1 Mbit chip).
- **Address decoders** that translate the address input into a word-line selection, enabling one row of cells at a time.
- **Sense amplifiers** that detect the small differential voltage on the bit lines during a read.
- **I/O multiplexers** that connect the selected columns to the data bus.

Because memory arrays are highly regular and repetitive, they are typically the target circuits used to qualify and test new semiconductor process nodes — which is why leading memory manufacturers such as Samsung are consistently at the forefront of semiconductor process technology.

10.2.2 Dynamic RAM (DRAM)

Definition 10.2.2 (DRAM). *Dynamic Random Access Memory* (DRAM) stores each bit as a charge on a capacitor, controlled by a single access transistor. The capacitor charge leaks over time, requiring periodic *refresh* operations to maintain data integrity. DRAM is slower and more power-hungry than SRAM but achieves a much higher storage density and lower cost per bit.

10.2.2.1 DRAM Cell Operation

A DRAM cell consists of one transistor (the pass transistor) and one capacitor:

- **Write ‘1’:** Enable the pass transistor; transfer charge onto the capacitor.
- **Write ‘0’:** Enable the pass transistor; discharge the capacitor.
- **Read:** Enable the pass transistor; a sense amplifier measures the capacitor charge to determine the stored logic value.

Warning: DRAM Read is Destructive

Measuring the charge on a DRAM capacitor via the sense amplifier effectively discharges it. Therefore, every read operation *destroys* the stored data, and the original value must be re-written immediately after each read (a *read-restore* cycle). Additionally, capacitor charge leaks naturally, so the entire DRAM array requires a *periodic refresh* even when idle.

10.2.2.2 Synchronous DRAM (SDRAM) and DDR Variants

Basic asynchronous DRAM is largely obsolete in modern systems. It has been superseded by *Synchronous DRAM* (SDRAM), which synchronises data transfer to a host-provided clock signal, enabling:

- Higher transfer rates through precise timing coordination.
- A *pipeline architecture* that allows overlapping of successive memory operations.

Modern systems use *Double Data Rate* (DDR) SDRAM variants (DDR, DDR2, DDR3, DDR4, DDR5), which transfer data on both the rising and falling edges of the clock, achieving sustained transfer rates exceeding 2 GT/s (giga-transfers per second).

Key Result: SRAM vs. DRAM Comparison

Property	SRAM	DRAM
Transistors per cell	4–6	1 (+ 1 capacitor)
Speed (access time)	Faster	Slower
Cost per bit	Higher	Lower
Density	Lower	Higher
Refresh needed?	No	Yes
Read destructive?	No	Yes
Typical use	Cache, registers	Main memory

10.3 Non-Volatile Memory

10.3.1 EPROM and EEPROM

The earliest erasable non-volatile memories used a *floating gate* transistor design:

- **EPROM** (Erasable Programmable ROM) — Programmed electrically; erased by exposing the chip to *ultra-violet (UV) light* through a transparent window in the package. Effective but operationally inconvenient (device must be removed from the circuit to erase).
- **EEPROM** (Electrically Erasable PROM, also written E²PROM) — Advances in semiconductor process technology reduced the gate oxide thickness sufficiently to allow the stored charge to be removed electrically (by applying a controlled high voltage), eliminating the need for UV light.

10.3.2 The Floating Gate Transistor (FGT)

The operating principle underlying EPROM, EEPROM, and Flash is the *Floating Gate Transistor* (FGT).

10.3.2.1 MOSFET Fundamentals

An N-channel MOSFET becomes conductive when the gate–source voltage V_{GS} exceeds a threshold voltage V_T :

$$I_D > 0 \iff V_{GS} > V_T.$$

When $V_{GS} > V_T$, electrons are attracted to the region beneath the oxide, forming a conductive *inversion layer* between the drain and source terminals.

10.3.2.2 Floating Gate Operation

In an FGT, a second conductive layer (the *floating gate*) is inserted between the control gate and the channel, sandwiched within the oxide. Because it is electrically isolated, any charge trapped in it persists even after power is removed — providing non-volatile storage.

Programming (writing a ‘0’): A large positive voltage (e.g. ~ 20 V, for illustration) is applied to the control gate. This causes electrons from the substrate to tunnel through the thin oxide and embed in the floating gate.

Erased state (storing a ‘1’): No excess electrons are present in the floating gate; the FGT behaves like a normal MOSFET with a relatively low threshold V_{T1} .

Programmed state (storing a ‘0’): Excess electrons in the floating gate add negative charge, raising the effective threshold voltage to $V_{T2} > V_{T1}$.

Reading: A read voltage V_{read} is applied to the control gate, chosen such that $V_{T1} < V_{\text{read}} < V_{T2}$:

- Erased state ($V_T < V_{\text{read}}$): FGT turns ON $\Rightarrow$ output = Logic ‘1’.
- Programmed state ($V_T > V_{\text{read}}$): FGT stays OFF $\Rightarrow$ output = Logic ‘0’.

Key Result: FGT Threshold Shift Encodes Data

Programming raises V_T ; erasing lowers it. A read voltage between V_{T1} (erased) and V_{T2} (programmed) discriminates the two states. An erased FGT array therefore contains all ‘1’s; programming writes ‘0’s into selected cells.

Warning: Flash Can Only Clear Bits, Not Set Them Individually

Programming in Flash can only change a cell from ‘1’ to ‘0’. Changing a ‘0’ back to ‘1’ requires an *erase*, which is always performed at *block level* (a block may be 64 KB, 128 KB, or 256 KB). Attempting to overwrite a single byte whose block contains ‘0’ bits requires: (1) read the entire block, (2) modify the byte in RAM, (3) erase the block, (4) reprogram the entire block.

10.3.3 Flash Memory

Flash memory is the dominant non-volatile semiconductor storage technology today. It builds on the FGT principle but allows erasure in larger blocks compared to EEPROM, achieving a lower cost per bit and higher write throughput. There are two principal architectures: NOR Flash and NAND Flash.

10.3.3.1 NOR Flash

In NOR Flash, cells are connected such that the bit line is driven low if *any* selected word line exceeds the programmed threshold — analogous to a NOR gate.

Key properties:

- **Execute In Place (XIP):** The processor can supply an address and receive data directly, just as with SRAM. This allows program code stored in NOR Flash to be executed without first copying it to RAM.
- **Random read:** Any byte can be read using only an address — no special command required for reads.

- **Byte/word programmable:** Individual bytes or words may be programmed (changed from '1' to '0'). However, erasure is always at block level.
- **Use case:** System memory for embedded systems (storing boot firmware, OS); can also be used as storage memory.

10.3.3.2 NAND Flash

In NAND Flash, cells are connected in series such that the bit line is driven low only when *all* selected word lines exceed the threshold — analogous to a NAND gate.

Key properties:

- **No XIP:** Data must be read a *page* at a time (a page is a fixed number of bytes). Access requires issuing a command, then the address, then reading the data — a multi-step sequence that precludes direct instruction execution.
- **Multiplexed bus:** Commands, addresses, and data share a single I/O bus, reducing pin count and enabling smaller, cheaper chips at the cost of protocol complexity.
- **Higher density, lower cost per bit:** Fewer inter-cell connections per bit allow NAND to be fabricated at greater density than NOR.
- **Use case:** Mass storage — USB flash drives, SD cards, SSDs, smartphones.

Key Result: NOR vs. NAND Flash

Property	NOR Flash	NAND Flash
Supports XIP	Yes	No
Read granularity	Random (byte/word)	Page-level
Bus interface	Separate Addr + Data	Multiplexed I/O
Cost per bit	Higher	Lower
Density	Lower	Higher
Typical use	Boot code / system memory	Mass storage

10.4 Storage Devices: HDD and SSD

10.4.1 Magnetic Hard Disk Drive (HDD)

A *Hard Disk Drive* (HDD) stores data by magnetising a thin ferromagnetic film on one or more rotating circular platters. Despite the rise of Flash-based storage, HDDs remain the dominant bulk storage medium in data centres due to their low cost per bit.

10.4.1.1 Mechanical Components

- **Platter** — The rotating disk coated with ferromagnetic recording medium. Most drives use multiple platters on a common spindle, with data stored on both surfaces of each platter.
- **Spindle** — A motor that rotates the platters at a constant rate, typically 5,400, 7,200, or 15,000 RPM.

- **Read/Write (R/W) Head** — Converts magnetic flux changes on the platter surface to electrical signals (read) or induces magnetic orientation changes (write). One R/W head is assigned to each platter surface.
- **Actuator Arm** — Carries the R/W head radially across the platter surface. Motion is produced by electromagnetic forces (voice coil actuator). The fly height of the R/W head above the platter surface is smaller than a fingerprint ridge — illustrating the extreme precision required.

10.4.1.2 Data Organisation

- **Track** — A concentric ring on one platter surface. Gaps between adjacent tracks reduce cross-track magnetic interference.
- **Sector** — A fixed-length arc of a track. The sector is the minimum addressable data unit. Each sector contains a header (synchronisation), data payload, and a trailer (error-correction code).
- **Cylinder** — The set of same-numbered tracks across all platter surfaces (stacked vertically). Used in legacy addressing.

10.4.1.3 HDD Transfer Rate and Timing Parameters

Four timing parameters characterise HDD access performance:

Parameter	Definition
Seek Time T_S	Time for the actuator arm to move the R/W head to the correct track. Often specified as an <i>average</i> by the manufacturer.
Rotational Delay T_R	Time for the disk to rotate until the R/W head is positioned at the start of the target sector.
Access Time T_A	$T_A = T_S + T_R$. Total time from request until the head is in position to begin reading.
Transfer Time T_T	Time to read or write the required data once the head is positioned.

Rotational delay formula. Since the R/W head may land anywhere on the track when seeking, the *average* rotational delay is:

$$T_{R,av} = \frac{0.5}{\text{RPS}} \text{ seconds,} \quad \text{where RPS} = \frac{\text{RPM}}{60}.$$

The factor of 0.5 arises because the best case is zero delay (head lands exactly on the target) and the worst case is a full revolution (head just missed the target), giving an average of half a revolution.

Transfer time formula. Let D_T = number of sectors per track, D_S = bytes per sector, and N = number of bytes to transfer:

$$T_T = \frac{N}{\text{RPS} \times D_T \times D_S} \text{ seconds.}$$

The denominator $\text{RPS} \times D_T \times D_S$ is the track throughput in bytes per second; dividing N by this gives the fraction of one revolution needed to read N bytes.

Total access time:

$$T_{\text{total}} = T_A + T_T = T_S + T_{R,av} + T_T.$$

Example 10.4.1 (HDD Transfer Rate Calculation). A magnetic HDD rotates at 15,000 RPM with average seek time $T_S = 4$ ms, track density $D_T = 500$ sectors/track, and sector density $D_S = 512$ bytes/sector. Find the total time to read a 3 KB file in consecutive sectors on the same track.

$$\begin{aligned} \text{RPS} &= \frac{15,000}{60} = 250 \text{ rev/s} \\ T_A &= 4 \text{ ms} + \frac{0.5}{250} = 4 \text{ ms} + 2 \text{ ms} = 6 \text{ ms} \\ T_T &= \frac{3072}{250 \times 500 \times 512} = \frac{3072}{64,000,000} \approx 48 \text{ } \mu\text{s} \\ T_{\text{total}} &= 6 \text{ ms} + 0.048 \text{ ms} \approx 6.048 \text{ ms.} \end{aligned}$$

Notice that $T_A \gg T_T$. If the file data were spread across different tracks, each track-to-track move incurs another T_A , severely degrading effective throughput — the motivation for *disk defragmentation*.

Exam Tip: HDD Timing Calculation Steps

Always convert RPM to RPS first. Then compute $T_{R,av} = 0.5/\text{RPS}$. Remember that N in the transfer time formula must be in *bytes* (not kilobytes); 3 KB = $3 \times 1024 = 3072$ bytes. Final answer should quote both T_A and T_T separately before summing.

10.4.1.4 Physical and Logical Layout

Physical layout refers to how data is actually arranged on the platter surface. Modern HDDs use *Zone Bit Recording* (ZBR): outer tracks are longer and can physically accommodate more sectors, so different zones have different numbers of sectors per track. This maximises recording density across the entire surface.

Logical layout is the abstract address space presented to the operating system. Two addressing schemes exist:

- **CHS (Cylinder–Head–Sector)** — Legacy scheme using the old constant-sectors-per-track physical model. Now obsolete due to capacity limitations.
- **LBA (Logical Block Addressing)** — Simple linear address space starting at LBA 0. The 48-bit LBA standard supports up to 2^{48} blocks, i.e. $2^{48} \times 512 = 2^{57}$ bytes ≈ 128 PB.

The HDD controller firmware translates between the standard LBA logical interface (used by all software) and the proprietary physical ZBR layout. This abstraction ensures software portability across HDDs from different manufacturers.

10.4.2 Solid State Drive (SSD)

An SSD replaces the mechanical HDD platters with a *NAND Flash memory array* (occasionally NOR Flash for specific applications). SSDs have no moving parts, making them faster, more power-efficient, quieter, and more robust to mechanical shock.

10.4.2.1 SSD Architecture

A typical SSD contains:

- **NAND Flash IC array** — the primary storage medium.

- **SSD Controller** — a microprocessor managing: logical-to-physical address translation, wear levelling, caching, and error correction.
- **External SRAM cache/buffer** — speeds up read and write operations by staging data in fast SRAM before committing to Flash.

10.4.2.2 SSD Challenges and Mitigation Techniques

The primary limitation of NAND Flash is its **finite program–erase (P/E) cycle endurance** (typically 3,000 to 1,000,000 cycles depending on cell type). Exceeding this limit causes permanent cell failure. Three key mitigation strategies are employed:

- **Wear Levelling** — The SSD controller distributes erase operations evenly across all blocks. This prevents any single block from being worn out disproportionately, extending the overall lifetime of the drive.
- **External RAM Buffer** — Data is first written to SRAM; only when the SRAM buffer is full (or on a flush event) is the data committed to Flash. This reduces the number of Flash write operations.
- **Error Correction Code (ECC)** — Detects and corrects bit errors that arise from cell wear, enabling continued operation even as some cells degrade.

Key Result: HDD vs. SSD Trade-offs

Property	HDD	SSD
Cost per bit	Lower	Higher
Speed	Slower (mechanical)	Faster
Power consumption	Higher	Lower
Robustness	Vulnerable to shock	Resistant (no moving parts)
Write endurance	Effectively unlimited	Limited P/E cycles
Noise	Audible	Silent

The single dominant reason HDD remains in widespread use is **cost**: HDD has a substantially lower cost per bit than SSD, which is decisive at the multi-petabyte scale of data centres.

10.5 Data Centre Storage

10.5.1 Storage Selection Criteria for Data Centres

Data centres operate at a scale where every design parameter has a large aggregate cost or engineering impact. The key criteria when selecting a storage element for data centre use are:

- **Cost** — The dominant criterion. A data centre may house exabytes of storage; even small differences in cost per bit translate to enormous absolute differences.
- **Speed** — Higher-speed storage improves application performance, particularly for database caching and virtual machine hosting (modern data centres do far more than simple file storage).

- **Power consumption** — Power costs are a major operational expense, both directly (electricity) and indirectly (cooling infrastructure). Data centres are often sited near natural cooling sources (e.g. large bodies of water) to reduce cooling costs.
- **Robustness** — High reliability reduces maintenance costs and improves service availability. Tolerance to mechanical vibration and shock is particularly important in dense rack environments.
- **Heat production** — Less heat means less cooling load and lower power consumption.
- **Physical size** — Smaller storage elements allow higher storage density per rack, improving space efficiency.

10.5.2 Why HDDs Still Dominate Data Centre Storage

On all criteria *except cost*, SSD outperforms HDD. Yet HDDs remain the dominant data centre storage medium because cost is overwhelmingly the most important criterion at data centre scale: the cost per bit of SSD is still substantially higher than HDD, and this difference is decisive when multiplied across petabytes of capacity.

Global storage shipment projections (IDC/Seagate data) confirm this: although SSD is gradually gaining market share, the total shipped *capacity* of HDD continues to grow, driven by the insatiable demand for low-cost bulk storage in data centres.

10.5.3 Heat Assisted Magnetic Recording (HAMR)

HDD manufacturers have continued to develop technologies to increase areal density — the amount of data storable per unit area of platter — in order to maintain their cost-per-bit advantage over SSD.

Definition 10.5.1 (Areal Density). *Areal density* is the number of data bits that can be stored per unit area of recording medium, typically expressed in terabits per square inch (Tbpsi).

A fundamental challenge in increasing areal density is the *superparamagnetic effect*: as magnetic grains are made smaller (to pack more bits into the same area), they become thermally unstable and may spontaneously flip their magnetic orientation, corrupting stored data. To counteract this, HDD manufacturers use recording media with *higher coercivity* — meaning the magnetic orientation is more stable. However, higher coercivity also means the recording head’s magnetic field is no longer strong enough to write to the media under normal conditions.

HAMR resolves this dilemma by attaching a small laser to the recording head. During a write operation:

1. The laser heats a tiny spot on the platter (~ 1 ns pulse) to a temperature that *temporarily* lowers the local coercive field below that of the write head.
2. The write head magnetises the heated spot, recording the bit.
3. Immediately after the heat pulse, the spot cools rapidly, and the high coercivity is restored — locking the bit’s magnetic orientation in a stable state.

Key Result: HAMR Enables Higher Areal Density

HAMR decouples the write process from storage stability by temporarily softening the recording medium only at the precise write location. Current projections suggest HAMR can achieve areal densities of 5 Tbpsi, enabling single drives with capacities exceeding 100 TB — preserving HDD’s cost-per-bit advantage well into the future.

Exam Tip: Connecting Memory Topics

When answering questions that compare storage technologies, always frame the answer around the specific *use case*: (1) If code must be executed in place, only SRAM, DRAM (with controller), or NOR Flash qualify as system memory. (2) If cost-per-bit is the dominant concern at large scale, HDD wins over SSD today. (3) If robustness, power, and speed matter more than cost, SSD wins. (4) NAND Flash (in SSD) requires wear levelling — never assume it behaves like SRAM for write endurance.

Chapter 11

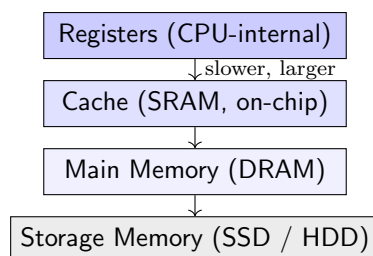
Cache Memory

11.1 Motivation and Overview

Modern CPUs operate in the GHz frequency range, while external main memory (DRAM) typically runs in the hundreds of MHz range. This speed mismatch means that the CPU spends a large fraction of its time stalled, waiting for data from main memory — making external memory the primary performance bottleneck of a computer system.

Cache is a small, fast memory placed between the CPU and main memory. Its purpose is to store recently or frequently used code and data *closer* to the CPU, so that most accesses can be served at near-CPU speed rather than at the slower main-memory speed. A well-designed cache can dramatically improve the effective memory access time of the whole system.

The hierarchy from fastest/smallest to slowest/largest is:



11.1.1 CPU Memory Access Sequence

When the CPU needs a piece of data it always queries the nearest memory level first:

1. The CPU sends a request to the cache.
2. **Cache hit**: the data is present — cache returns it immediately. (*Best case.*)
3. **Cache miss**: the data is absent — the CPU queries main memory.
4. If the data is also absent from main memory, a further request goes to disk (storage memory). (*Worst case.*)
5. Once the data is located at any level, the required data *and a block of its neighbours* are loaded into cache (exploiting spatial locality).

Key Result: Cache as a Buffer

Cache hides the latency of main memory from the CPU. The entire effectiveness of cache rests on the *Principle of Locality*: programs tend to access the same data and nearby data repeatedly, so a small cache can serve a large fraction of all memory requests.

11.2 Principle of Locality

Definition 11.2.1 (Principle of Locality). Programs exhibit *locality*: once a byte of code or data is accessed, it is highly probable that nearby bytes (in address space) or the *same* byte (over time) will be accessed soon.

There are two complementary types:

Spatial Locality Code and data that reside at *nearby addresses* are likely to be accessed together. This justifies fetching data in *blocks* rather than individual bytes: when a miss occurs, the entire block containing the needed byte is copied into cache, so subsequent accesses to adjacent addresses are cache hits.

Temporal Locality Recently accessed code or data is likely to be accessed *again soon*. This governs cache *replacement policy*: when a block must be evicted to make room, the least recently used block is evicted (it is the least likely to be needed again).

Example 11.2.2 (Locality in a Loop). Consider the loop:

```
1 for (i = 0; i < 10; i++)  
2   y[i] = a[i] * x[i];
```

- **Spatial:** Accessing `a[0]` makes it very likely that `a[1]`, `a[2]`, ... will be accessed next — justifying the block-fetch strategy.
- **Temporal:** The loop body is executed 10 times, so the same instructions are fetched from cache on each iteration rather than from main memory.

Exam Tip: Transfer is Always in Blocks

Data transfer between main memory and cache always happens in *whole blocks* (cache lines), never one byte at a time. This is a direct consequence of spatial locality. Do not confuse the *block size* (e.g. 4 bytes or 16 bytes) with the individual byte requested by the CPU.

11.3 Cache Mapping Scheme

The *cache mapping scheme* determines which cache location a given main memory block maps to. Three schemes exist: Direct Mapped, Set Associative, and Fully Associative. This course covers only **Direct Mapped Cache**; the other two are treated in Advanced Computer Architecture.

11.3.1 Key Terminology

Cache Block (Cache Line) The basic unit of storage in the cache. A cache with block size B bytes holds B bytes in each block. All transfers between main memory and cache move one whole block.

Block Index (Cache Block Number) The integer index $0, 1, \dots, N - 1$ identifying which block of the N -block cache is being referred to.

Offset The byte position within a block, ranging from 0 to $B - 1$. The number of bits required for the offset field is $\log_2 B$.

Tag Bits from the main memory address left over after allocating for Block Index and Offset. The tag uniquely identifies which main memory block occupies a given cache block.

11.3.2 Direct Mapped Cache

In a direct-mapped cache with N cache blocks, main memory block number X (not the full address, but the block number) maps deterministically to cache block:

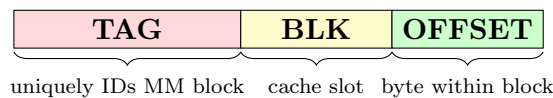
$$Y = X \bmod N.$$

This gives each main memory block a *fixed* home in the cache. If two frequently accessed blocks share the same home, they thrash each other out of cache (a limitation of direct mapping).

11.3.2.1 Address Field Partitioning (TAG : BLK : OFFSET)

Every main memory address is partitioned into three contiguous fields, always computed from the *least significant end*:

1. **OFFSET** (LSBs): $\log_2(\text{block size in bytes})$ bits.
2. **BLK** (middle bits): $\log_2(\text{number of cache blocks})$ bits.
3. **TAG** (MSBs): remaining bits = total address bits – BLK bits – OFFSET bits.



11.3.3 Cache Hit and Miss Detection

When the CPU requests main memory address A :

1. Partition A into TAG, BLK, OFFSET using the mapping format.
2. Go to cache block BLK .
3. Compare the stored tag of that cache block with the TAG field of A .
4. **Cache Hit**: tags match — the correct block is in cache. Retrieve the byte at position $OFFSET$.
5. **Cache Miss**: tags differ — the block needed is not present. The corresponding main memory block is fetched and written into cache block BLK , overwriting whatever was there. Then the CPU retrieves the byte.

Example 11.3.2 (Cache Hit and Miss). Using the same system from Example 11.3.1 (mapping format 4:2:2 in the lecture's smaller example with 8-bit addresses):

- **Request 0x8C**: Binary 1000 11 00 $\Rightarrow$ TAG = 0x8, BLK = 3, OFFSET = 0. Cache block 3 holds tag 0x8: *match* $\Rightarrow$ **Cache Hit**.
- **Request 0x84**: Binary 1000 01 00 $\Rightarrow$ TAG = 0x8, BLK = 1, OFFSET = 0. Cache block 1 holds tag 0x9: *mismatch* $\Rightarrow$ **Cache Miss**. The MM block containing 0x84–0x87 is fetched into cache block 1.

11.4 Cache Replacement Policy

For direct-mapped cache the replacement destination is uniquely determined by the block number ($X \bmod N$), so no policy decision is needed. In *set-associative* and *fully associative* caches (covered in Advanced Computer Architecture), there is a choice of which existing block to evict when all candidate slots are occupied. Two common policies are:

Least Recently Used (LRU) Evicts the block that has been unused for the longest time. Conforms closely to temporal locality and achieves high efficiency; however, it requires maintaining a per-block access-history record, adding hardware cost and slowing down the cache.

First-In, First-Out (FIFO) Evicts the block that has been in cache the longest, regardless of how recently it was accessed. Simpler to implement than LRU but can evict frequently-used blocks, resulting in lower efficiency.

11.5 Cache Write Policy

Cache design so far has focused on read operations. Write operations require additional policy decisions. A cache block that the CPU has written to but whose corresponding main memory block has *not* yet been updated is called a **Dirty Block**.

11.5.1 Write-Hit Policies

A *write hit* occurs when the CPU writes to an address currently present in cache.

Write-Through The CPU writes simultaneously to both the cache block and the corresponding main memory location on every write. Guarantees cache–memory *coherence* at all times, but generates heavy traffic on the system bus (memory bus) because every write propagates to main memory.

Write-Back (Copyback) The CPU writes only to the cache block. The main memory is updated *only when the dirty cache block is evicted*. Reduces bus traffic significantly, but the cache module must track whether each block is dirty (using a *dirty bit*) to know whether a write-back is required on eviction. There is a transient window of *incoherence* between cache and main memory.

11.5.2 Write-Miss Policies

A *write miss* occurs when the CPU writes to an address *not* currently in cache.

Write-Allocate (Fetch-on-Write) The corresponding main memory block is first loaded into cache, then the write is performed on the cached copy (following the write-hit policy). Future accesses to the same block are likely to be cache hits. Pairs naturally with Write-Back.

Write-No-Allocate The CPU writes directly to main memory without bringing the block into cache. The cache is not updated. Simpler, but subsequent accesses to the same block will still miss. Pairs naturally with Write-Through.

Key Result: Write Policy Combinations

In practice, the two commonly paired combinations are:

- **Write-Back + Write-Allocate:** fewer bus transactions, more complex dirty-bit management.
- **Write-Through + Write-No-Allocate:** simpler coherence, higher bus traffic.

11.6 Effective Access Time (EAT)

Definition 11.6.1 (Effective Access Time). The *effective access time* (EAT) is the weighted average memory access time experienced by the CPU, taking into account both the probability of a cache hit and the access times of each memory level.

Let:

- H = cache hit rate (fraction of accesses that are cache hits, $0 \leq H \leq 1$).
- Access_C = cache access time.
- Access_{MM} = main memory access time.

Assuming cache and main memory are accessed *sequentially* (i.e., on a miss, the cache is checked first, then main memory — accesses do not overlap):

$$\text{EAT} = H \cdot \text{Access}_C + (1 - H) \cdot (\text{Access}_C + \text{Access}_{MM}). \quad (11.1)$$

This simplifies to:

$$\text{EAT} = \text{Access}_C + (1 - H) \cdot \text{Access}_{MM}.$$

Warning: EAT Formula Applies to READ Only

Equation (11.1) applies only to *read* operations. Write operations are more complex because the effective penalty depends on the combination of write-through/write-back and write-allocate/write-no-allocate policies in use.

Example 11.6.2 (EAT Calculation). A system has main memory access time 200 ns, cache access time 10 ns, and a cache hit rate of 99%.

$$\text{EAT} = 0.99 \times 10 \text{ ns} + 0.01 \times (10 + 200) \text{ ns} = 9.9 \text{ ns} + 2.1 \text{ ns} = 12 \text{ ns}.$$

Without cache, every access would take 200 ns. The cache reduces the effective access time to 12 ns — a 16.7× improvement. This illustrates how a cache with a high hit rate and a 20× speed advantage can dominate the overall EAT.

Exam Tip: EAT Steps

(1) Identify H , Access_C , and Access_{MM} . (2) Apply Equation (11.1). (3) Confirm the assumption that accesses are *sequential* (the problem should state this). (4) The miss penalty is $\text{Access}_C + \text{Access}_{MM}$, *not* just Access_{MM} , because the CPU still pays the cache access time even on a miss.

Chapter 12

Virtual Memory Management

12.1 Memory Address Space

Definition 12.1.1 (Address Space). An *address space* is a range of discrete addresses corresponding to a physical or logical entity — for example, the virtual address space of a program, the physical address space of main memory, or the logical block address space of a hard disk.

Different entities within a computer each maintain their own address space with their own addressing scheme:

- **Program (Virtual Address Space):** Addresses generated by the compiler and assigned by the OS. Each program has its own independent virtual address space.
- **System/Main Memory (Physical Address Space):** Addresses of actual DRAM locations.
- **Storage Memory (Logical Block Addressing, LBA):** Addresses of 512-byte sectors on an HDD or SSD (see Chapter 9).

The same piece of data (e.g. a video file) may be stored at *different addresses* in each of these spaces simultaneously. *Address translation* — performed by the Operating System (OS) — bridges these different address spaces transparently.

12.1.1 Virtual vs. Physical Address

- The addresses a program uses are called **virtual addresses**. They are generated by the compiler and allocated by the OS.
- The addresses of actual locations in main memory are called **physical addresses** (also called “main memory addresses” or “system memory addresses” — all synonymous in this course).
- Virtual addresses are typically *different* from the physical addresses where the data actually resides.
- The OS is responsible for translating virtual to physical addresses. This translation is implemented in hardware and/or software.

12.2 Virtual Memory Management

The OS manages virtual memory by:

1. Maintaining a *per-program* virtual address space. Each running program sees its own private virtual memory, independent of other programs.

2. Loading only the *subset* of a program's code and data that is required at any given moment into physical main memory. The complete program resides in storage memory (HDD/SSD).
3. Performing *address translation* from virtual to physical addresses on every memory access.

12.2.1 Advantages of Virtual Memory

- **Isolation and Protection:** Each program's virtual address space is isolated from every other program's. The OS can prevent one program from reading or corrupting another program's memory.
- **Efficient Sharing:** Multiple programs share the same physical memory simultaneously. The OS multiplexes physical memory across programs, only keeping the actively used portions of each in RAM.
- **Programs Larger than Physical Memory:** The OS can run programs whose total combined size exceeds physical memory by swapping pages between main memory and storage memory as needed.
- **Contiguous Logical View:** The OS maps virtual address space to *fragmented* physical frames in a way that appears contiguous to the program. This greatly simplifies programming — the programmer need not manage physical memory fragmentation.

12.3 Paging: Address Mapping Scheme

Two address mapping schemes exist in industry: **Paging** (fixed-size blocks) and **Segmentation** (variable-size blocks). This course covers **paging with a single-level page table** only.

Definition 12.3.1 (Page and Frame). In a paging system:

- The virtual address space is divided into equal-sized units called **pages**.
- The physical address space is divided into equal-sized units called **frames**.
- Page size $\equiv$ Frame size (always, by definition).

The OS maps each virtual page to a physical frame. The mapping is recorded in a data structure called the **Page Table**.

12.3.1 The Page Table

Each entry in the page table contains at minimum:

Field	Meaning	Note
Page #	Virtual page number	Index into the table
Frame #	Physical frame the page is mapped to	Used in address translation
Valid Bit (VB)	1 = page is currently in physical memory	0 = page must be fetched from storage

If VB = 1, the page's data is in the physical frame given by Frame #. If VB = 0, a **page fault** is generated — the OS must load the required page from storage memory into an available physical frame, update the page table, and retry the memory access.

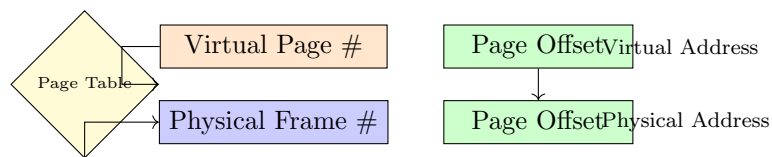
12.4 Virtual-to-Physical Address Translation

The process of translating a virtual address to a physical address in a paging system is entirely analogous to the TAG:BLK:OFFSET split used in cache mapping (Chapter 11), but the two fields here are **Virtual Page Number** and **Page Offset**.

12.4.1 Address Field Partitioning

1. Convert the virtual address to binary.
2. **Offset field** (LSBs): $\log_2(\text{page size in bytes})$ bits. *The offset is preserved unchanged in the physical address.*
3. **Virtual Page Number** (VPN, remaining MSBs): used to index the page table.

The physical address is formed by:



The offset bits pass through unchanged. Only the upper bits (VPN) are replaced by the corresponding Physical Frame Number read from the page table.

Key Result: Address Translation Steps

1. Convert virtual address to binary.
2. Compute offset bits: $\log_2(\text{page size})$.
3. Extract VPN from the upper bits; extract offset from the lower bits.
4. Look up VPN in the page table:
 - Valid Bit = 1: replace VPN with the Frame # to form the physical address.
 - Valid Bit = 0: **page fault** — OS loads the page from storage, updates the table, retries.
5. Physical address = [Frame #][Offset] concatenated in binary.

12.4.2 Worked Example: Paging System

Example 12.4.1 (Address Translation in an 8K/4K System). **System:** Virtual address space = 8 KB, Physical address space = 4 KB, Page size = 1 KB.

Derived parameters:

$$\text{Offset bits} = \log_2(1024) = 10.$$

$$\text{Virtual address bits} = \log_2(8192) = 13.$$

$$\text{Physical address bits} = \log_2(4096) = 12.$$

$$\text{VPN bits} = 13 - 10 = 3 \Rightarrow 2^3 = 8 \text{ virtual pages.}$$

$$\text{Frame \# bits} = 12 - 10 = 2 \Rightarrow 2^2 = 4 \text{ physical frames.}$$

Suppose the page table is:

Page #	Frame #	Valid Bit
0	–	0
1	3	1
2	0	1
3	–	0
4	–	0
5	1	1
6	2	1
7	–	0

Translate virtual address 0x1553:

$$0x1553 = (1\ 0101\ 0101\ 0011)_2.$$

VPN (upper 3 bits) = $101_2 = 5$. Offset (lower 10 bits) = $01\ 0101\ 0011_2$.

Page 5 has Valid Bit = 1 and maps to Frame 1.

Physical address = $01_2 \parallel 01\ 0101\ 0011_2 = (01\ 0101\ 0101\ 0011)_2 = 0x0553$.

Translate virtual address 0x0FA0:

$$0x0FA0 = (0\ 1111\ 1010\ 0000)_2.$$

VPN = $011_2 = 3$. Page 3 has Valid Bit = 0.

⇒ **Page Fault.** The OS must load virtual page 3 from storage memory into a free physical frame, update the page table, and re-execute the access.

Warning: Page Fault ≠ Error

A page fault is *not* a program error. It is a normal OS mechanism: the required page simply is not in main memory at that moment. The OS services the fault transparently by loading the page from disk and resuming the program.

12.5 Integration of Cache and Virtual Memory

A real system implements *both* cache and virtual memory simultaneously. When the CPU accesses a virtual address, the typical sequence (assuming the cache uses *physical addresses*) is:

1. Translate the virtual address to a physical address using the page table.
2. Check the cache using the physical address (cache hit or miss evaluation using the TAG:BLK:OFFSET scheme from Chapter 11).
3. On a cache hit: retrieve data from cache.
4. On a cache miss: fetch the block from main memory into cache, then return the data.

If the cache uses *virtual addresses* for indexing, the CPU queries the cache *before* address translation, but must still perform translation on a cache miss to fetch data from main memory.

12.6 Translation Lookaside Buffer (TLB)

Definition 12.6.1 (TLB). A *Translation Lookaside Buffer* (TLB) is a small, fast cache for page table entries, residing inside the processor. It stores a subset of the most recently or frequently used virtual-page-to-physical-frame mappings.

The page table can be several megabytes in size and must reside in main memory, making every address translation an expensive main-memory access. The TLB avoids this overhead: if the required mapping is in the TLB (a **TLB hit**), the physical frame number is obtained at internal-memory speed, far faster than a main memory access.

Warning: TLB $\neq$ Data Cache

The TLB caches *address translation information* ($\text{VPN} \mapsto \text{Frame \#}$) only. It does not store program code or data. The data cache (Chapter 11) stores actual code and data. They are separate hardware structures serving different purposes.

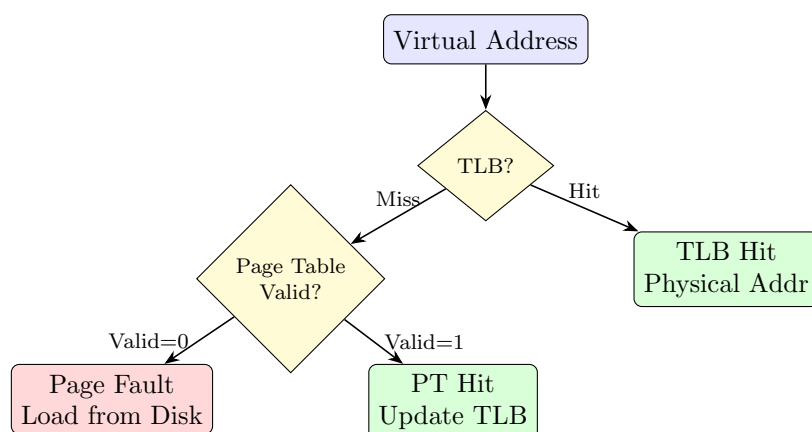
12.6.1 TLB Lookup: Three Scenarios

Starting from a virtual address:

TLB Hit The VPN is found in the TLB. The corresponding Frame # is read directly, the physical address is formed, and data is accessed. Fast path — no main memory access needed for translation.

TLB Miss (Page Table Hit) The VPN is not in the TLB but is found with Valid Bit = 1 in the main-memory page table. The Frame # is obtained from main memory, the TLB is updated with this entry, and the physical address is formed. One extra main memory access for translation.

Page Fault The VPN is neither in the TLB nor valid in the page table. The OS loads the required page from storage into a free physical frame, updates the page table, and optionally updates the TLB. Slowest path — disk access required.



Key Result: TLB as a Cache for Page Table

The TLB functions like a cache (Chapter 11) but for page table entries rather than data. A high TLB hit rate means virtual-to-physical translation is performed almost entirely at processor speed, eliminating the cost of main-memory page table lookups for the vast majority of accesses.

Exam Tip: Three-Level Address Lookup Summary

Know all three access paths (TLB Hit / TLB Miss with Page Table Hit / Page Fault) and be able to state what happens at each step. Pay attention to whether the TLB is updated after a TLB miss — this depends on the VM design and may be tested explicitly.

Chapter 13

Communication and User Interface

This chapter provides a general overview of the common wired and wireless communication technologies used in computers—specifically USB and HDMI for wired, and Wi-Fi and Bluetooth for wireless—as well as the user-interface devices that transfer real-world data into and out of the digital domain: keyboards, mice, capacitive touch screens, cameras, microphones, displays, and audio speakers. The discussion here is intentionally at a conceptual level; detailed protocol-layer analysis is beyond the scope of SC1006.

13.1 Wired Communication Interfaces

13.1.1 Universal Serial Bus (USB)

Definition 13.1.1 (Universal Serial Bus). The *Universal Serial Bus (USB)* is a serial bus standard designed to provide a single, unified interface for connecting computer peripheral devices—keyboards, mice, printers, disk drives, network adapters, digital cameras, and many others. It effectively replaced a large number of legacy interface buses such as the RS-232 serial (COM) port and the parallel printer port.

USB has evolved substantially since its introduction. USB 1.0 supported a maximum transfer rate of 12 Mbps; the latest USB 3.2 standard supports up to 20 Gbps. The basic USB connector has four pins: VBUS (+5 V power supply), Data+ and Data− (the differential data pair), and Ground.

13.1.1.1 USB Topology and Host Model

USB employs a *tiered-star topology*. At the centre of every star sits the **USB Host**, which is typically integrated into the computer’s chipset. Peripheral devices (mice, keyboards, etc.) connect either directly to the Host or indirectly through intermediate **USB Hub** devices.

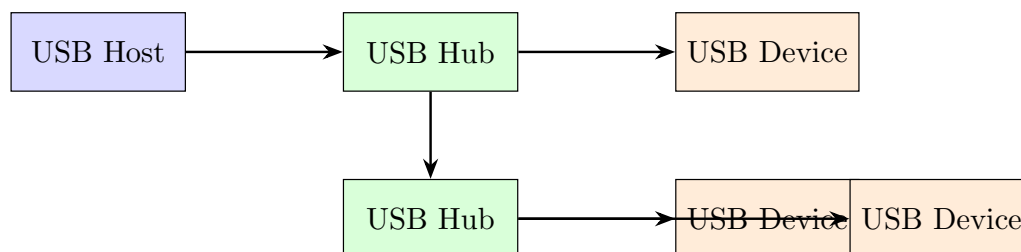


Figure 13.1: USB tiered-star topology. Hubs extend the bus; all transactions are initiated by the Host.

Several important properties follow from the host-centric design:

- All data transactions are *initiated by the USB Host*. Data flowing into the Host is called an **IN** transaction; data flowing out is an **OUT** transaction.

- The Host assigns a unique address to each connected device so that it can direct traffic to the correct peripheral.
- Power can be supplied by the Host or Hub (*bus-powered*), or the device may have its own supply (*self-powered*).

13.1.1.2 USB Enumeration and Device Classes

When a USB device is first connected, it undergoes an *enumeration* process: the device and the Host exchange capability and requirement information—for example, the device reports its Vendor ID, Product ID, device class, and required bandwidth. The Host cannot communicate with the device until enumeration is complete.

After enumeration the Host checks whether it has a suitable *device driver* matching the USB **device class** that the peripheral belongs to. Common USB device classes include:

- **Human Interface Device (HID)** — used for mice and keyboards.
- **Communication Device Class (CDC)** — used to implement a Virtual COM Port (e.g. Arduino boards, MSP432 LaunchPads).
- **Mass Storage Class (MSC)** — used to interface with external USB HDDs and SSDs.
- **USB Audio Class** — used to stream audio to USB headsets and microphones.

Exam Tip: USB Host-Initiated Transactions

Remember that *all* USB data transactions are initiated by the Host, not the peripheral device. The Host polls peripherals according to their endpoint descriptors; even an “interrupt” transfer type is actually Host-pollled at a guaranteed frequency.

13.1.2 High-Definition Multimedia Interface (HDMI)

Definition 13.1.2 (HDMI). *HDMI (High-Definition Multimedia Interface)* is a proprietary audio/video interface for transmitting uncompressed video data and compressed or uncompressed digital audio data from a source device (e.g. a display controller in a computer) to a compatible receiver such as a monitor, digital television, or digital audio device.

Beyond audio/video transport, HDMI includes a **CEC (Consumer Electronics Control)** capability that allows connected HDMI devices to control each other, enabling the user to operate multiple devices with a single remote control.

Three commonly used HDMI connector types exist: **Type A** (standard), **Type C** (Mini), and **Type D** (Micro). The original HDMI v1.0/1.1 specification supported a transfer rate of 3.96 Gbps, sufficient for 1080p video at 60 fps. The latest HDMI v2.1 achieves 42.6 Gbps, enabling 8K-resolution video.

13.2 Wireless Communication

13.2.1 ISM Radio-Frequency Bands

Wireless standards used in computing almost universally operate within the *Industrial, Scientific and Medical (ISM)* radio-frequency bands—a set of “royalty-free” frequency allocations. Some ISM bands are global (available worldwide) while others are restricted to specific geographical regions. The two most important worldwide ISM bands are **2.4 GHz** and **5.8 GHz**. Multiple wireless technologies—including Wi-Fi, Bluetooth, ZigBee, and proprietary low-power radio—all share these bands, which can lead to mutual interference.

13.2.2 Wi-Fi (IEEE 802.11)

Definition 13.2.1 (Wi-Fi). *Wi-Fi* is a family of wireless networking technologies based on the IEEE 802.11 family of standards, commonly known as “Wireless LAN.” Wi-Fi operates in the 2.4 GHz and 5.8 GHz ISM bands.

Wi-Fi supports two common topologies:

- **Infrastructure Mode** — a star topology with an Access Point (AP) or router at the centre, to which all end devices connect.
- **Ad-hoc Mode** — a peer-to-peer connection between devices with no central access point.

The typical Wi-Fi transmission range lies between 20 m and 150 m, depending on factors such as transmission frequency, transmission power, and interference. The latest standard, 802.11ax (Wi-Fi 6), achieves transfer rates of up to approximately 10 Gbps.

13.2.3 Bluetooth

Definition 13.2.2 (Bluetooth). *Bluetooth* is a short-range wireless standard designed primarily for low-data-rate transmission with an emphasis on low power consumption. It operates in the 2.4 GHz ISM band and uses a star topology.

Common Bluetooth applications include wireless headsets, smart wearables, and Bluetooth mice. The maximum transmission range is approximately 100 m, but devices typically operate at 10–20 m to minimise power draw. Transfer rates are on the order of Kbps to Mbps.

The Bluetooth standard defines two distinct protocols: **Bluetooth Classic (BT Classic)** and **Bluetooth Low Energy (BLE)**. These are based on completely different network-layer protocols and are *not mutually compatible*. However, most modern Bluetooth hosts are “Dual Mode” and can connect to both BT Classic and BLE devices.

13.2.4 Factors Affecting Wireless Transmission Range

Three principal factors govern the effective range of a wireless link:

1. **Transmission power** — higher power directly extends the range.
2. **Transmission frequency** — higher-frequency signals experience greater attenuation as they propagate through air or other media; all else being equal, a higher-frequency signal has a shorter range than a lower-frequency one.
3. **Interference** — many commonly adopted standards (Wi-Fi, Bluetooth, ZigBee) share the 2.4 GHz ISM band and can interfere with one another. The closer the transmitters, the stronger the interference. Even household microwave ovens radiate energy in the 2.4 GHz range.

13.2.5 Mitigating Interference

The “2.4 GHz ISM band” actually spans a range of frequencies from approximately 2.4 to 2.5 GHz. The Wi-Fi standard divides this band into sub-bands called *channels*, each 22 MHz wide. Two common interference-mitigation strategies are:

- **Channel selection** — choosing a channel different from those used by neighbouring networks to reduce overlap.

- **Frequency hopping** — continuously hopping from one channel to another so that any given collision affects only a brief portion of the transmission. Bluetooth uses frequency hopping.

Key Result: Wireless Range Trade-offs

Higher frequency $\Rightarrow$ higher data rate capability but shorter range due to increased atmospheric attenuation. Higher transmission power extends range but increases energy consumption and potential interference with other devices sharing the same band.

13.3 User Interface Devices — Input

13.3.1 Keyboard

A keyboard is a user input device for transmitting characters (alphabets, numbers, special symbols, etc.) to the computer. Internally, a small microcontroller detects key presses and sends their corresponding **ASCII codes** to the host PC. Wired keyboards today predominantly use the USB interface and are enumerated as **HID Keyboard** devices (recall Section 13.1.1). Wireless keyboards typically operate in the 2.4 GHz ISM band using either Bluetooth or a proprietary RF protocol with a USB dongle receiver.

13.3.2 Mouse

A mouse is a user pointing device that tracks its position by mechanical or optical means. The core data reported to the PC are the (x, y) coordinate deltas and the states of the left/right buttons; mice with additional features may report scroll-wheel data or extra button states.

Position data is reported to the PC periodically at what is called the *scan/polling rate*. A typical mouse polls at 125 Hz; gaming mice can reach 1000 Hz or higher, yielding lower input latency and smoother cursor tracking.

Another key parameter is the **Dot-Per-Inch (DPI)**, which measures how finely the mouse can track physical movement—a higher DPI means greater sensitivity to small physical movements. Connection technologies mirror those of keyboards (USB HID for wired, 2.4 GHz ISM for wireless).

13.3.3 Capacitive Touch Interface

Definition 13.3.1 (Capacitive Touch Sensing). A *capacitive touch* pad or button is modelled electrically as a capacitor. When a conductive element—such as a human finger—approaches or contacts the pad, the effective capacitance of the system increases because additional capacitors (C_{E1-F} , C_{F-E2}) appear in parallel with the existing capacitance C_0 .

The capacitance of a parallel-plate structure is given by

$$C = \frac{\epsilon_0 \epsilon_r A}{d},$$

where ϵ_r is the relative permittivity (dielectric constant) of the material between the plates, A is the plate area, and d is the separation. Since air has $\epsilon_r \approx 1$ and most other materials have $\epsilon_r > 1$, placing any dielectric (e.g. gloves, plastic, liquid) between the finger and the pad changes the sensed capacitance. This explains why capacitive touch screens do not respond well to gloved fingers or when the screen surface is wet—the dielectric changes shift the measured capacitance away from the calibrated “bare finger” baseline.

A dedicated processor reads the capacitance pattern and calculates the exact position of contact.

Common Misconception: Touch Screens and Gloves

Capacitive touch screens are calibrated under the assumption of a bare human finger. Wearing gloves or having a wet screen alters the dielectric environment and degrades touch accuracy—it is not simply that the screen “cannot detect” a gloved finger, but rather that the capacitance change deviates from the expected profile.

13.4 User Interface Devices — Sensors

13.4.1 Camera

Modern cameras on phones and PCs typically use a **CMOS camera module** composed of four sub-modules:

1. **Lens** — an optical element that focuses real-world objects onto the image sensor.
2. **Image Sensor** — predominantly CMOS-based; it transduces photons (light) into analog electrical signals. An on-chip ADC then converts these analog voltages to digital values.
3. **Image Signal Processor (ISP)** — a specialised processor that handles Auto Focus, Auto Exposure, Auto White Balance, image format conversion (e.g. between YUV and RGB colour spaces), and image post-processing/compression.
4. **Interface I/O** — reformats the processed image data from the ISP for delivery to the host processor over a bus such as MIPI CSI or USB.

Historically, image sensors were based on *Charge-Coupled Device (CCD)* technology; the CCD was invented by George Smith and Willard Boyle at Bell Labs, who received the Nobel Prize in Physics in 2009 for this work. Today, CMOS sensors dominate due to lower power consumption, easier integration, and reduced manufacturing cost.

13.4.2 Microphones

The two most popular microphone technologies are **electret condenser microphones (ECM)** and **MEMS (micro-electro-mechanical system) microphones**. Both types use the variation in capacitance caused by diaphragm displacement under sound pressure to transduce sound waves into electrical signals. The relationship $V = Q/C$ means that a change in the plate separation d (caused by sound pressure) changes the capacitance C , which in turn changes the voltage V across the capacitor.

13.4.2.1 Electret Condenser Microphone (ECM)

In an ECM, the diaphragm is made of an *electret*—a material with a permanent fixed surface charge. This electret is placed near a conductive pick-up plate, forming a capacitor with the air gap as the dielectric. Sound pressure waves displace the diaphragm, changing the plate separation and hence the capacitance. Since the charge Q on the electret is fixed, the voltage across the capacitor varies as $V = Q/C$.

13.4.2.2 MEMS Microphone

A MEMS microphone operates on the same capacitive transduction principle but uses a micro-machined structure consisting of a movable membrane and a fixed back-plate with acoustic holes. Sound enters through the holes, moves the membrane, and modulates the air gap between

the two conductive plates. Unlike the ECM, the charge needed to sense the capacitance change is supplied by an external *charge pump* circuit integrated into the MEMS package.

The transduced signal from either type is inherently analog, but an ADC can be added (often integrated on the same ASIC in MEMS microphones) to provide a digital output directly.

13.5 User Interface Devices — Output

13.5.1 Liquid Crystal Display (LCD)

Definition 13.5.1 (Liquid Crystal). A *liquid crystal* is an organic substance that possesses both a liquid form and a crystal molecular structure. Its rod-shaped molecules can maintain an ordered orientation in a particular direction, and an applied electric field can control this orientation.

In an LCD, the liquid crystal layer sits between two polarisers oriented at 90° to each other. The key mechanism is as follows:

1. With **no applied voltage**, the liquid crystal molecules are arranged in a twisted configuration that gradually rotates the polarisation of incoming light by 90° . The rotated light passes through the second polariser $\Rightarrow$ the pixel appears *bright*.
2. With an **applied voltage**, the electric field straightens the molecules so that they no longer twist the light. The unrotated polarised light is blocked by the second polariser $\Rightarrow$ the pixel appears *dark*.
3. Intermediate voltages produce partial twisting, allowing control of brightness levels.

For a **colour LCD**, each pixel is associated with three sub-pixel colour filters (Red, Green, Blue). By varying the molecular orientation in each sub-pixel independently, the amount of light reaching each colour filter is controlled, enabling the display to reproduce a full spectrum of colours. Thin-Film Transistors (TFTs) at each sub-pixel act as switches to hold the required voltage level.

Key Result: LCD Light-Gating Principle

An LCD does not emit light; it *gates* light from a backlight source. The liquid crystal layer acts as a voltage-controlled polarisation rotator placed between crossed polarisers. Pixel brightness is set by the degree of molecular twist, which is governed by the applied electric field.

13.5.2 Audio Speakers

An audio speaker transduces electrical energy into sound energy. The operating principle relies on the electromagnetic interaction between a permanent magnet and a voice coil:

1. An audio signal (alternating current) flows through the **voice coil**, which is attached to a **speaker cone** (diaphragm).
2. The voice coil sits in the magnetic field of a **permanent magnet**. The varying current creates a changing electromagnetic field around the coil, causing it to move back and forth within the permanent magnet's field.
3. This mechanical motion is transferred to the speaker cone, which pushes and pulls the surrounding air, creating **sound waves**.

The conversion path is therefore: electrical current → electromagnetic force on voice coil → mechanical vibration of cone → sound pressure waves. Note that this process is the reverse of how a dynamic microphone works—in fact, a speaker and a dynamic microphone are essentially the same transducer operated in opposite directions.

Exam Tip: Transduction Summary for I/O Devices

Be able to state the transduction principle for each device covered:

- **Camera sensor:** photons → electrons → voltage → digital value (via ADC).
- **Microphone:** sound pressure → diaphragm displacement → capacitance change → voltage change.
- **LCD:** electrical field → molecular orientation → polarisation rotation → light gating.
- **Speaker:** electrical current → electromagnetic force → cone vibration → sound waves.

Chapter 14

Computer Arithmetic and Number Representation

This chapter covers the representation and arithmetic of signed integers in binary, focusing on the two's complement system used by virtually all modern processors. It then introduces fixed-point and floating-point number systems, culminating in the IEEE 754 standard for single- and double-precision floating-point representation.

14.1 Signed Integer Representation

14.1.1 Positive and Negative Numbers in Binary

To represent signed integers, computer systems allocate the **Most Significant Bit (MSB)** to indicate the sign of the number: MSB = 0 denotes a positive value, and MSB = 1 denotes a negative value. The remaining bits encode the magnitude, but the interpretation of those bits depends on the representation scheme. The most commonly used scheme in modern computing is *two's complement*.

14.1.2 Two's Complement Representation

Definition 14.1.1 (Two's Complement). In *two's complement* representation, positive numbers are stored as ordinary unsigned binary. To obtain the representation of a negative number, one *inverts* (flips) all bits of the corresponding positive value and then *adds one* to the result, discarding any carry out of the MSB.

Example 14.1.2 (Negating +106 in 8-bit Two's Complement).

$$\begin{array}{rcl} +106 & = & 0110\ 1010_2 \\ \text{Invert} & = & 1001\ 0101_2 \\ \text{Add 1} & = & 0000\ 0001_2 \\ \hline -106 & = & 1001\ 0110_2 \end{array}$$

The MSB of the result is 1, confirming the value is negative.

14.1.2.1 Two's Complement to Decimal Conversion

The decimal value of an n -bit two's complement number $b_{n-1} b_{n-2} \dots b_1 b_0$ is computed using a *weighted sum* in which the MSB carries a **negative** weight:

$$\text{Value} = -b_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i.$$

Example 14.1.3 (8-bit Two's Complement to Decimal). Convert 10001110_2 to decimal using the table of weights:

-2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
-128	64	32	16	8	4	2	1
1	0	0	0	1	1	1	0
-128	0	0	0	8	4	2	0

$-128 + 8 + 4 + 2 = -114$. Therefore 10001110_2 in 8-bit two's complement equals -114 in decimal.

Key Result: Two's Complement Range

An n -bit two's complement integer can represent values in the range

$$-2^{n-1} \leq x \leq 2^{n-1} - 1.$$

For example, 8-bit two's complement spans -128 to $+127$.

14.1.3 Carry vs. Overflow

When performing two's complement addition, a *carry out* of the MSB does not necessarily indicate an error. In the subtraction $48 - 19 = 48 + (-19) = 29$, converting 19 (00010011_2) to its two's complement gives 11101101_2 . Adding $00110000_2 + 11101101_2$ produces 00011101_2 with a carry out of 1—yet the 8-bit result (29) is correct.

Definition 14.1.4 (Overflow in Two's Complement). *Overflow* in two's complement arithmetic occurs when the result of an operation falls outside the representable range. It is detected by inspecting the MSBs of the operands and the result:

- **Addition** ($R = A + B$): overflow if $\text{MSB}(A) = \text{MSB}(B)$ and $\text{MSB}(R) \neq \text{MSB}(A)$.
- **Subtraction** ($R = A - B$): overflow if $\text{MSB}(A) \neq \text{MSB}(B)$ and $\text{MSB}(R) \neq \text{MSB}(A)$.

The following table summarises the four overflow scenarios (using 4-bit examples):

Expression	Result	Carry?	Overflow?	Correct?
$0100 (+4) + 0010 (+2)$	$0110 (+6)$	No	No	Yes
$0100 (+4) + 0110 (+6)$	$1010 (-6)$	No	Yes	No
$1100 (-4) + 1110 (-2)$	$1010 (-6)$	Yes	No	Yes
$1100 (-4) + 1010 (-6)$	$0110 (+6)$	Yes	Yes	No

Common Misconception: Carry Means Overflow

For **unsigned** numbers, a carry out of the MSB *does* indicate overflow. For **signed** (two's complement) numbers, carry can occur without overflow and vice versa. Always check the *overflow flag*, not the carry flag, when working with signed arithmetic.

14.1.4 Sign Extension

Definition 14.1.5 (Sign Extension). *Sign extension* is the process of converting a smaller two's complement operand to a larger bit-width by copying the sign bit (MSB) into all of the new higher-order bit positions. This preserves the numeric value.

For example, extending 8-bit values to 16 bits:

- $00101000_2 (+40) \rightarrow 00000000\ 00101000_2 (+40)$. The MSB is 0, so zeros are prepended.

- $10000000_2 (-128) \rightarrow 11111111\ 10000000_2 (-128)$. The MSB is 1, so ones are prepended.
- $10011010_2 (-102) \rightarrow 11111111\ 10011010_2 (-102)$.

Exam Tip: Sign Extension Rule

When sign-extending a two's complement number, simply replicate the MSB into all new upper bits. This works for both positive and negative values. Do *not* zero-extend a signed number—that changes its value if it is negative.

14.1.5 Multi-Precision Arithmetic

When operands are wider than the ALU (e.g. a 64-bit addition on a 32-bit processor), the computation is split across multiple ALU operations. The lower-order words are added first with an ordinary **ADD** instruction, which produces a carry flag. The upper-order words are then added using an **ADC** (Add with Carry) instruction that incorporates the carry from the previous step.

Example 14.1.6 (64-bit Addition on a 32-bit ALU). Add $0x00000001\text{FFFFFFFF}$ and $0x00000001\ 00000001$:

```

1  ADD R0, R0, R1 ; lower 32 bits: 0xFFFFFFFF + 0x00000001
2                  ; = 0x00000000 with Carry = 1
3  ADC R2, R2, R3 ; upper 32 bits: 0x00000001 + 0x00000001 + 1
4                  ; = 0x00000003

```

The 64-bit result is $0x00000003\ 00000000$.

14.2 Fixed-Point and Floating-Point Number Systems

14.2.1 Range and Precision

Definition 14.2.1 (Range and Precision). The *range* of a number system is the interval between the smallest and largest representable values ($\max^-$ to $\max^+$). The *precision* is the amount of information used to represent each number—equivalently, how closely spaced the representable values are on the number line.

For a given number of bits, there is a fundamental trade-off between range and precision: increasing the range (by allocating more bits to the integer/exponent part) necessarily decreases the precision (fewer bits remain for the fractional/mantissa part), and vice versa.

14.2.2 Fixed-Point Representation

Definition 14.2.2 (Fixed-Point). In *fixed-point* representation, the position of the radix (binary) point is fixed at a predetermined location within the bit pattern. Bits to the left of the radix point represent integer powers of 2 ($2^0, 2^1, \dots$), while bits to the right represent fractional powers ($2^{-1}, 2^{-2}, \dots$).

For example, an 8-bit fixed-point format with 4 integer bits and 4 fractional bits (denoted 4.4) has positional weights $2^3, 2^2, 2^1, 2^0, 2^{-1}, 2^{-2}, 2^{-3}, 2^{-4}$. The representable range (unsigned) is 0 to 15.9375 with a uniform resolution of $2^{-4} = 0.0625$ across the entire range.

Key properties of fixed-point:

- **Uniform precision:** the spacing between consecutive representable values is constant everywhere.

- **Limited range:** allocating more bits to the integer part increases range but reduces precision, and vice versa.
- **Simple hardware:** fixed-point arithmetic uses the same integer ALU, making it fast and inexpensive.

14.2.3 Floating-Point Representation

Definition 14.2.3 (Floating-Point). A *floating-point* number is represented by three fields: a **sign** bit (S), a **mantissa** (also called the *significand* or *fraction*), and an **exponent**. The value is

$$\text{Value} = (-1)^S \times \text{Mantissa} \times \text{Base}^{\text{Exponent}}.$$

The exponent effectively “floats” the radix point to the correct position, hence the name.

The key consequence of this representation is that the density of representable numbers is **non-uniform**: numbers near zero are packed closely together (high precision), while numbers far from zero are spaced further apart (lower precision). This matches many practical applications where the greatest precision is needed for small values.

14.2.3.1 Normalisation

Without normalisation, the same value can have multiple representations (e.g. 1.00×10^1 , 0.10×10^2 , 0.01×10^3 all represent 10). *Normalisation* requires exactly one non-zero digit before the radix point, eliminating synonymous representations and maximising the number of significant digits stored in the mantissa.

In binary, since the only non-zero digit is 1, a normalised binary floating-point number always has the form $1.f_1f_2f_3 \dots \times 2^e$. Because the leading 1 is always present, it need not be stored explicitly—this is known as the *hidden bit* or *implicit leading 1*, and it effectively gives one extra bit of precision for free.

14.2.3.2 Underflow

Definition 14.2.4 (Underflow). *Underflow* occurs when a computed value is too close to zero to be represented by the smallest normalised number. The *underflow region* is the gap between zero and the smallest positive (or largest negative) normalised value.

Both overflow (value too large in magnitude) and underflow (value too small in magnitude) can cause program crashes or silent loss of accuracy if not handled properly.

14.3 IEEE 754 Floating-Point Standard

The **IEEE 754** standard, found in virtually every computer since 1980, unified floating-point representation across platforms, simplifying both portability and algorithm development. It defines two primary formats:

- **Single Precision** (32 bits): 1-bit sign + 8-bit exponent + 23-bit fraction.
- **Double Precision** (64 bits): 1-bit sign + 11-bit exponent + 52-bit fraction.

14.3.1 Normalised Numbers

For normalised IEEE 754 numbers, the value is computed as:

$$\text{Value} = (-1)^S \times (1.F)_2 \times 2^{E-\text{Bias}}, \quad (14.1)$$

where:

- S is the sign bit (0 = positive, 1 = negative).
- E is the stored (biased) exponent, ranging from 1 to $2^k - 2$ for normalised numbers (i.e. 00000001₂ to 11111110₂ for single precision).
- $F = f_1f_2f_3\ldots$ is the stored fraction. The hidden leading 1 is *not* stored but is implicit.
- Bias = 127 for single precision and 1023 for double precision.

Example 14.3.1 (Single Precision to Decimal Conversion). Decode the 32-bit pattern: 0 10110010 111000000000000000000000.

Sign	$S = 0$ (positive)
Exponent	$E = 10110010_2 = 178$; $E - 127 = 51$
Fraction	$F = 111_2$; $1+F = (1.111)_2 = 1.875$
Value	$+1.875 \times 2^{51}$

14.3.2 Representable Range (Single Precision)

The smallest-magnitude normalised single-precision number has $E = 1$ and $F = 0$:

$$\pm 1.0 \times 2^{1-127} = \pm 2^{-126} \approx \pm 1.175 \times 10^{-38}.$$

The largest-magnitude normalised number has $E = 254$ and $F = 111\ldots 1$:

$$\pm (2 - 2^{-23}) \times 2^{127} \approx \pm 2^{128} \approx \pm 3.403 \times 10^{38}.$$

14.3.3 IEEE 754 Encoding Summary

The exponent field values $E = 0$ (all zeros) and $E = 255$ (all ones, for single precision) are reserved for special encodings:

Mode	Sign	Exponent	Fraction
Normalised	0/1	00000001 to 11111110	Anything
Denormalised	0/1	00000000	Non-zero
Zero	0/1	00000000	0000...0000
Infinity ($\pm\infty$)	0/1	11111111	0000...0000
NaN	0/1	11111111	Non-zero

Denormalised (subnormal) numbers use $E = 0$ with a non-zero fraction and an implicit leading 0 (instead of 1), allowing representation of values in the underflow gap between zero and the smallest normalised number.

Key Result: IEEE 754 Value Formula

For normalised single-precision numbers: $\text{Value} = (-1)^S \times 1.F \times 2^{E-127}$, where E ranges from 1 to 254. The bias converts the unsigned stored exponent into a signed effective exponent ranging from -126 to $+127$.

14.4 Fixed-Point vs. Floating-Point Comparison

Given the same total bit-width (e.g. 32 bits), the two systems offer different trade-offs:

	Fixed-Point (32-bit)	Floating-Point (IEEE 754 SP)
Max range	$\approx 2^{32}$ (unsigned)	$\approx 2 \times 2^{128}$
Best precision	2^{-32} (all bits fractional)	$< 2^{-126}$ (near zero)
Precision	Uniform across range	Non-uniform; finest near zero

Floating-point provides a vastly larger range and superior precision for small numbers, which is typically where the highest precision is needed. However, fixed-point has the advantage of *uniform precision* across its entire range—the spacing between representable values never changes. In floating-point, precision degrades significantly at the extremes of the range, which may be undesirable for certain algorithms (e.g. in digital signal processing or financial calculations where uniform resolution is required).

Exam Tip: Fixed vs. Floating-Point Trade-offs

Be prepared to explain the key differences: floating-point offers larger range and non-uniform precision (best near zero), while fixed-point offers smaller range but uniform precision. Know when each is preferable and be able to compute the range and resolution of a given fixed-point format.

14.5 Computer Arithmetic Considerations in Coding

When writing code—whether in a high-level language or in assembly—it is essential to understand how the underlying arithmetic hardware affects the correctness and accuracy of computations. Two central issues arise: *overflow* (the result exceeds the representable range) and *loss of precision* (bits are discarded during rounding or truncation). This section examines how the four basic arithmetic operators interact with these issues and presents practical rules for maximising computational accuracy.

14.5.1 Effects of the Four Arithmetic Operators

Definition 14.5.1 (Arithmetic Overflow). Arithmetic overflow occurs when the result of an operation falls outside the range [Min, Max] of the number system in use (determined by the data type, register width, or memory word size).

Addition and Subtraction. Addition increases the magnitude of the result; subtraction may increase or decrease it depending on the sign of the operands. When performed repeatedly, the accumulated result will eventually overflow at the minimum or maximum end of the representable range. The programmer must be aware of the range of the data type used in a high-level language, or the width of the registers and memory when coding in assembly.

Multiplication. Multiplication in binary is analogous to an arithmetic left shift—each multiplication by two shifts the binary representation one position to the left. As a result, repeated multiplication rapidly increases magnitude and can overflow the representable range, subject to the same considerations as addition and subtraction.

Division. Division in binary is analogous to an arithmetic right shift. Unlike the other three operators, division *reduces* the magnitude of the result, moving it further away from the extremes of the representable range. However, right-shifting causes the least significant bit(s)

to be truncated, leading to a *loss in precision*. Division therefore does not cause overflow but introduces rounding error.

Warning: Division Destroys Precision

Division truncates LSBs, permanently losing information. Once precision is lost, subsequent operations cannot recover it. To preserve accuracy, always perform division as late as possible in a computation.

14.5.2 Effects of Rounding

Definition 14.5.2 (Rounding). Rounding is the process of removing least significant bit(s) so that a result can fit within the available representable bits. The width limit may be imposed by the register size, data type, or number format.

Rounding may be *round-up* or *round-down* to the nearest representable number. Since the rounded value is only an approximation of the true result, a *rounding error* is incurred with each rounding operation.

Example 14.5.3 (Decimal Rounding Error). Consider rounding a floating-point number to 2 decimal digits in the significand:

$$1.686 \times 10^1 \longrightarrow 1.69 \times 10^1$$

The rounding error is $0.004 \times 10^1 = 0.04$, which is small individually but accumulates over many operations.

To mitigate rounding error, processors commonly use *intermediate registers* whose width is larger than the regular data registers. This allows intermediate computations to proceed at higher precision, and rounding is applied only when the final result is written back to a standard-width register, thereby reducing the total accumulated rounding error.

14.5.3 Handling Numbers with Different Magnitudes

When adding or subtracting floating-point numbers of vastly different magnitudes, the smaller operand must be *aligned* (shifted) so that both exponents match. If the smaller number is much smaller than the larger one, this alignment may shift all of its significant digits out of the representable precision, effectively reducing it to zero.

Example 14.5.4 (Order of Evaluation Matters). Suppose we wish to compute $1.00 \times 10^0 + 1.00 \times 10^0 + 1.00 \times 10^0 + 1.00 \times 10^0 + 1.23 \times 10^3$ using a 3-digit significand. Adding the five small values first yields 5.00×10^0 , which, when aligned and added to 1.23×10^3 , produces:

$$0.005 \times 10^3 + 1.230 \times 10^3 = 1.235 \times 10^3 \xrightarrow{\text{round}} 1.24 \times 10^3.$$

If instead we added each small value directly to 1.23×10^3 one at a time, each 1.00×10^0 would align to 0.001×10^3 and be rounded away, yielding only 1.23×10^3 —an inferior result.

Key Result: Evaluation Order for Accuracy

When summing numbers of mixed magnitudes, **accumulate values of similar (small) magnitude first** before combining with large-magnitude operands. This allows the small values to contribute meaningfully to the final result rather than being rounded away during alignment.

14.5.4 Rules of Thumb for Maximising Accuracy

The two central issues in computer arithmetic—overflow and precision loss—can be mitigated by following a set of practical guidelines:

1. **Accumulate small-magnitude operands first.** Adding or subtracting values of similar small magnitude before combining with large values preserves their contribution (see Example 14.5.4).
2. **Know the representable range.** Be aware of the limits imposed by the data type (e.g. `int` vs. `long`), number format (fixed-point vs. floating-point), and register or memory width. This determines when overflow becomes a risk.
3. **Check for overflow.** Where feasible, insert overflow checks into the computation. Many languages and architectures provide flags or exceptions for arithmetic overflow.
4. **Perform division last.** Since division truncates LSBs and permanently destroys precision, delay all division operations until the final step of a computation whenever possible.

Exam Tip: Ordering Operations

A common exam question presents an expression and asks which evaluation order maximises accuracy. Remember: multiply and add first (these preserve precision), divide last (this loses precision), and accumulate small values together before combining with large ones (this prevents alignment loss).

Chapter 15

Processor Pipeline

Modern processors improve performance by overlapping the execution of multiple instructions through a technique called *pipelining*. This chapter introduces pipeline architecture, analyses its efficiency, and examines three types of pipeline conflicts—resource conflicts, data dependency conflicts, and branch conflicts—together with their resolution strategies.

15.1 Review: Sequential Instruction Execution

In a simple (non-pipelined) CPU, each instruction passes through three stages sequentially:

1. **Fetch (F)** — retrieve the instruction from memory using the Program Counter.
2. **Decode (D)** — the control unit interprets the opcode and identifies operands.
3. **Execute (E)** — the ALU performs the required operation.

Each stage occupies one clock cycle, and the next instruction cannot begin until the current instruction has completed all three stages. Consequently, executing n instructions on a simple CPU requires $3n$ clock cycles. For example, three instructions require $3 \times 3 = 9$ clock cycles.

Warning: Sequential Bottleneck

In a sequential CPU the Fetch, Decode, and Execute hardware units sit idle for two out of every three cycles. Pipelining addresses this by keeping all units active simultaneously.

15.2 Pipeline Architecture

15.2.1 Basic Concept

Definition 15.2.1 (Pipeline). Pipelining is the technique of partitioning instruction execution into simpler stages and assigning independent hardware resources to each stage so that multiple stages can operate simultaneously on different instructions.

In a 3-stage pipeline (F–D–E), once the pipeline is filled, a new instruction enters the Fetch stage every cycle while earlier instructions progress through Decode and Execute. Three instructions therefore complete in only 5 cycles instead of 9:

Pipelining is possible because the Fetch, Decode, and Execute operations use *independent resources*: Fetch uses the external buses, Decode uses the instruction decoder, and Execute uses the ALU. Since these units do not interfere with one another, they can operate concurrently on different instructions.

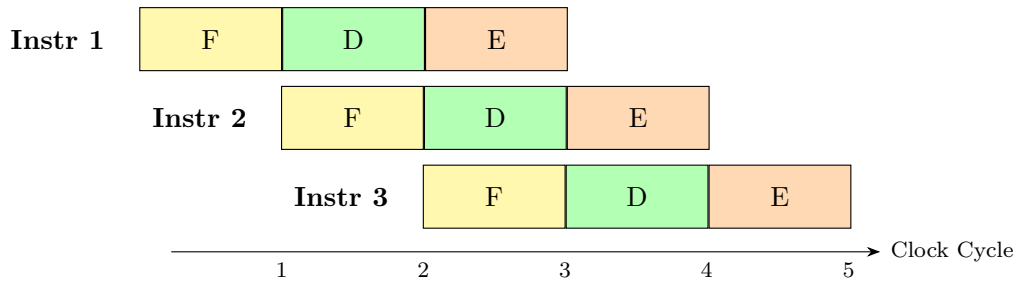


Figure 15.1: Three-stage pipeline timing diagram: 3 instructions complete in 5 cycles.

Key Result: Pipeline Speedup

For an k -stage pipeline executing n instructions, the total number of cycles is $k + (n - 1)$, compared with $k \cdot n$ for sequential execution. As n grows large the pipeline approaches an ideal throughput of **one instruction per clock cycle**.

15.2.2 Pipeline Efficiency

The efficiency of a pipeline is maximised when the pipeline is *fully filled*—that is, every stage is actively processing an instruction. Consider a 4-stage pipeline with stages Fetch (F), Decode (D), Execute (E), and Store (S). It takes 4 cycles to fill the pipeline and release the first instruction. After the pipeline is filled, however, one instruction completes every cycle, giving the effect of executing one instruction per clock cycle.

More complex application processors commonly employ 10 or more pipeline stages to further increase clock frequency, though deeper pipelines also increase the penalty when conflicts arise.

15.3 Pipeline Conflicts

Pipeline conflicts (also called *hazards*) are events that disrupt the smooth flow of instructions through the pipeline. They typically cause one of two outcomes: a temporary *stall* (halt) of the pipeline, or a *flush* in which partially processed instructions are discarded and the pipeline must be refilled. Both outcomes increase total execution time and reduce effective throughput.

Three sources of pipeline conflict are examined: resource conflicts, data dependency conflicts, and branch conflicts.

15.3.1 Resource Conflict (Structural Hazard)

Definition 15.3.1 (Resource Conflict). A resource conflict occurs when two instructions in different pipeline stages attempt to access the *same hardware resource* during the same clock cycle.

Example 15.3.2 (Memory Contention). In a 4-stage pipeline (F–D–E–S), suppose Instruction 1 reaches the Store stage and needs to write a result back to memory in the same cycle that Instruction 4 enters the Fetch stage and needs to read an instruction from the same memory. Both cannot use the memory bus simultaneously, creating a resource conflict.

Resource conflicts are resolved by providing sufficient independent resources for each pipeline stage. Modern pipeline processors typically include multiple internal buses (data bus, instruction bus, peripheral bus), separate instruction and data caches (Harvard architecture), and dedicated control and processing units so that all stages can operate simultaneously without contention.

Exam Tip: Resolving Resource Conflicts

The key solution to resource conflicts is **resource duplication**—for instance, using separate instruction memory and data memory (or caches) so that Fetch and Store can access memory simultaneously without conflict.

15.3.2 Data Dependency Conflict (Data Hazard)

Definition 15.3.3 (Data Dependency Conflict). A data dependency conflict arises when the source operand of the current instruction is the destination operand of a prior instruction that has not yet completed its Store stage. The current instruction reads a stale (old) value instead of the updated one.

This is not an issue in a sequential CPU because each instruction fully completes before the next begins, so registers always hold up-to-date values. In a pipeline, however, the overlapping execution means a dependent instruction may reach its Execute stage before the prior instruction has stored its result.

Example 15.3.4 (Data Dependency in a 4-Stage Pipeline). Consider the following code with initial register values $R1 = 0x2$, $R2 = 0x0$, $R3 = 0x5$:

```
1 ADD R2, R2, R1 ; R2 = R2 + R1 = 0x0 + 0x2 = 0x2
2 SUB R3, R3, R2 ; R3 = R3 - R2 (should use R2 = 0x2)
```

In a 4-stage pipeline (F–D–E–S), when SUB reaches the Execute stage (cycle 4), ADD is simultaneously in the Store stage. The updated value of $R2$ (0x2) has not yet been written back, so SUB reads the old value $R2 = 0x0$ and computes $R3 = 0x5 - 0x0 = 0x5$ instead of the correct $R3 = 0x5 - 0x2 = 0x3$.

The exact number of instructions that must separate a producer and consumer to avoid a data hazard depends on the pipeline depth and design.

15.3.2.1 Resolution: Stalling the Pipeline

Hardware circuitry can detect data dependencies by comparing the destination register identifier in the Execute stage with the source register identifier(s) in the Decode stage. When a match is found, the dependent instruction is *stalled* at the Decode stage until the prior instruction completes and the correct value is available. In the example above, stalling the SUB instruction for one cycle allows ADD to finish its Store stage, after which SUB reads $R2 = 0x2$ and correctly computes $R3 = 0x3$.

15.3.2.2 Resolution: Inserting NOP Instructions

Alternatively, the *compiler* can detect data dependencies during compilation and insert explicit NOP (No Operation) instructions between dependent instructions. The NOP occupies a pipeline slot without performing any useful work, effectively delaying the dependent instruction by one cycle so that the updated value is available by the time it reaches the Execute stage. This approach yields the correct result but increases total execution time because an extra instruction must pass through the pipeline.

Warning: Stall vs. NOP

Stalling is a *hardware* mechanism—the pipeline control logic detects the hazard at run time and pauses the dependent stage. NOP insertion is a *software/compiler* mechanism—the compiler inserts dummy instructions at compile time. Both introduce a one-cycle delay (for a 4-stage FDES pipeline with adjacent dependent instructions), but NOPs increase the total instruction count in the program.

15.4 Branch Conflicts (Control Hazards)

15.4.1 Branch Instruction Behaviour

A branch instruction (e.g. BEQ, BNE) performs two operations that both require computation: evaluating the branch condition (e.g. checking the Zero flag) and calculating the branch target address using the ALU. These operations naturally occur during the Execute (E) stage of the pipeline.

The problem is that while the branch instruction is being decoded and executed, the processor has already fetched one or more subsequent instructions into the pipeline. If the branch is *taken*, these instructions are incorrect and must be discarded (*flushed*). The number of cycles wasted by this flushing is called the **branch delay**, and the pipeline slots occupied by the discarded instructions are called **delay slots**.

Definition 15.4.1 (Branch Delay). The branch delay is the number of clock cycles wasted when the pipeline must be flushed due to a taken branch. It equals the number of pipeline stages between the Fetch of the instruction after the branch and the stage at which the branch decision is resolved.

Example 15.4.2 (Two-Cycle Branch Delay). In a 4-stage pipeline where the branch decision is made in the Execute stage, by the time the branch target is known, two subsequent instructions (I and J) have already been fetched and partially processed. If the branch is taken, both must be discarded, resulting in a 2-cycle branch delay.

15.4.2 Reducing Branch Delay

Branch delay can be reduced by moving the branch decision *earlier* in the pipeline. By introducing an additional adder in the Decode stage to compute the branch target address and resolving the branch condition at the Decode stage instead of Execute, the branch outcome is known one stage earlier. In this configuration, only one instruction (the one fetched immediately after the branch) needs to be discarded if the branch is taken, reducing the branch delay to **one cycle**.

15.4.3 Delayed Branching

Definition 15.4.3 (Delayed Branching). Delayed branching is a pipeline strategy in which the processor *always executes* the delay slot instruction(s) immediately following a branch, regardless of whether the branch is taken or not. No instructions are ever discarded, eliminating wasted cycles.

Delayed branching can be viewed as a configurable feature of the processor:

Delayed branching disabled. When the branch is taken, the delay slot instructions are discarded to preserve program correctness, at the cost of wasted cycles (branch delay).

Delayed branching enabled. The delay slot instructions are always executed. However, if the original delay slot instructions (I and J) would produce incorrect results when the branch is taken, the programmer or compiler must **rewrite the code** to fill the delay slots with *independent instructions* instead.

15.4.3.1 Requirements for Delay Slot Instructions

The instructions placed in delay slots must satisfy several conditions:

1. **Independence from the branch decision.** They must not modify any register or flag that affects the branch condition. For example, if a BEQ instruction depends on the Zero flag, the delay slot instruction must not alter the Zero flag (recall that all instructions are assumed to update condition flags by default, i.e. ADD behaves as ADDS).
2. **Always executed in the original program.** They must be instructions that would have been executed regardless of whether the branch is taken or not. This typically means they are instructions that appear *before* the branch in the original program sequence.
3. **Loop context.** If the branch is part of a loop, the delay slot instructions should be sourced from within the loop body, since they will execute on every iteration.
4. **Fallback.** If no suitable independent instruction can be found, the delay slot must be filled with a NOP to preserve correctness.

Example 15.4.4 (Code Rewriting for Delayed Branching).

<pre> 1 ; Original code: 2 Instr 1 3 Instr 2 4 Instr 3 5 BEQ Next 6 Instr I 7 Instr J 8 Next: Instr K </pre>	<pre> ; Rewritten code: Instr 1 BEQ Next Instr 2 ; delay slot 1 Instr 3 ; delay slot 2 Instr I Instr J Next: Instr K </pre>
--	---

If Instructions 2 and 3 are independent of the branch decision, they can be moved *after* the BEQ instruction to fill the delay slots. The processor executes them during the branch delay period, achieving zero wasted cycles.

Key Result: Delayed Branching

When delayed branching is enabled and the delay slots are filled with independent instructions, the branch delay is effectively **zero**—no cycles are wasted. If independent instructions cannot be found, NOPs must fill the delay slots, and the performance benefit is lost.

15.4.4 Dynamic Branch Prediction

Definition 15.4.5 (Dynamic Branch Prediction). Dynamic branch prediction uses hardware circuitry to *guess* the outcome of a conditional branch before it is resolved. A **branch history table** stores the predicted target address and the predicted outcome (taken/not taken) for each branch instruction encountered.

The prediction mechanism works as follows:

1. When a branch instruction is fetched, the processor looks up the branch history table using the branch instruction's address.

2. If an entry exists, the processor speculatively fetches instructions from the predicted target address.
3. When the branch is resolved during the Execute (or Decode) stage:
 - If the prediction was **correct**, execution continues normally with no wasted cycles.
 - If the prediction was **incorrect**, the speculatively fetched instructions are flushed (wasting cycles), and the prediction bit and target address in the table are updated for future reference.

Exam Tip: Branch Prediction vs. Delayed Branching

Branch prediction and delayed branching are two distinct strategies for mitigating branch delay. Prediction is a *hardware* approach that guesses the branch outcome, while delayed branching is a *software/architectural* approach that restructures the code so delay slot instructions are always useful. Both can reduce or eliminate branch delay, but prediction incurs a penalty when the guess is wrong, whereas delayed branching requires finding suitable independent instructions.

15.5 Note on ARM Instructions in Pipeline Examples

The pipeline examples in this module use ARM assembly instructions for illustration, but with the following simplifying assumptions:

- The pipeline structure discussed is **not** the actual ARM Cortex pipeline.
- All ARM instructions are assumed to occupy exactly **one word** and each pipeline stage takes exactly **one clock cycle**.
- The ‘S’ suffix is enabled by default on all instructions—i.e. every instruction updates the condition flags (e.g. ADD behaves as ADDS, MOV as MOVS).

15.6 Real-World Connection: ARM Cortex-M3/M4

The ARM Cortex-M3 and M4 processors use a **3-stage pipeline** (Fetch, Decode, Execute) with a branch speculation mechanism. When a branch instruction reaches the Decode stage, the processor speculatively fetches the branch destination instruction in parallel. During the Execute stage, the branch is resolved:

- If the branch is *not taken*, the next sequential instruction is already available.
- If the branch is *taken*, the branch target instruction has been pre-fetched during the Decode stage and is available immediately when the decision is made.

This speculation mechanism reduces branch delay to near zero in many cases, illustrating how real-world processors combine pipeline design with predictive hardware to maintain high throughput.